SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

# Django

## Intro & Design Pattern

### Python

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 목 차


## Web Application

- 클라이언트와 서버
- Frontend & Backend

## Framework

- Web Framework
- Django Framework

## 가상 환경

- 가상 환경 생성 및 활성화
- 의존성 패키지
- 의존성 관리: 환경의 일치
- 가상 환경 주의사항

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 목 차


## Django 프로젝트

- 프로젝트 생성 및 서버 실행

## Django Design Pattern

- Design Pattern
- 프로젝트와 앱
- 프로젝트 및 앱 구조

## 요청과 응답

- Django에서의 요청과 응답

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 목 차

## 참고

- 가상 환경 생성 루틴
- Django 관련
- Python 패키지 설치법
- render 함수
- MTV 디자인 패턴 정리
- Trailing Comma
- 프레임워크의 규칙 및 설계 철학

# 학습 시작



## 멋진 웹 페이지를 만들었습니다!

- 다른 사람들에게도 보여주고 싶다면 필요한 게 무엇일지 알아봅시다.

1. 웹 브라우저는 [ ]로서 인터넷 상의 컴퓨터에 [ ]을 보냅니다.
2. 이 [ ]을 받은 [ ]는 [ ]에 따라 알맞은 [ ]을 보냅니다.
3. 이러한 형태로 제공되는 서비스를 [ ]이라고 하며, 이를 빠르게 개발할 수 있도록 도와주는 도구가 [ ] 입니다.

- 자신이 만든 웹 페이지를 인터넷을 이용해 공유할 수 있습니다.
- 인터넷을 이용한 서비스가 어떤 형태로 동작하는지 이해할 수 있습니다.

# 학습 목표



- ✓ 웹 애플리케이션이 무엇인지 이해한다.
- ✓ 클라이언트 - 서버 구조를 설명할 수 있다.
- ✓ 가상 환경을 통해 서로 다른 프로젝트의 의존성을 관리할 수 있다.
- ✓ Django 프로젝트를 시작할 수 있다.
- ✓ Django를 활용해 사용자 브라우저에 HTML을 전송할 수 있다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# Web Application

## Web application (web service) 개발

- 인터넷을 통해 사용자에게 제공되는 소프트웨어 프로그램을 구축하는 과정
- 다양한 디바이스(모바일, 태블릿, PC 등)에서 웹 브라우저를 통해 접근하고 사용할 수 있음

네이버를 비롯한 포털, 쿠팡과 같은 인터넷 쇼핑몰부터
Google Docs와 같은 웹 문서 작업 도구 등 - 인터넷을 통해 접할 수 있으며
이를 통해 제공되는 서비스를 통틀어 Web Service라고 부릅니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

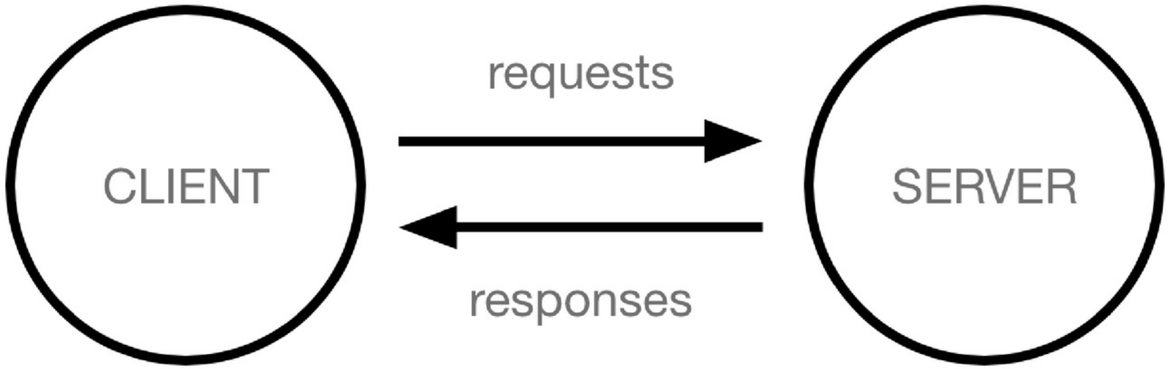
# 클라이언트와 서버

## 웹은 어떻게 동작할까?

# 웹의 동작 방식

우리가 컴퓨터 혹은 모바일 기기로 웹 페이지를 보게
될 때까지 무슨 일이 일어날까?

## 클라이언트 - 서버 구조




<그림1_Django_Intro & Design Pattern_클라이언트 서버 구조>

- Client - 서비스를 요청하는 주체
  - 사용자의 웹 브라우저, 모바일 앱
- Server - 클라이언트의 요청에 응답하는 주체
  - 웹 서버, 데이터베이스 서버

일반적인 웹 서비스에서는
클라이언트가 페이지를 달라고 “요청”할 경우
서버가 페이지를 “응답”해 줍니다.

## 우리가 웹 페이지를 보게 되는 과정 (1/4)

1. 웹 브라우저(클라이언트)에서 ‘google.com’을 입력 후 Enter

google.com 입력!

Client

Server

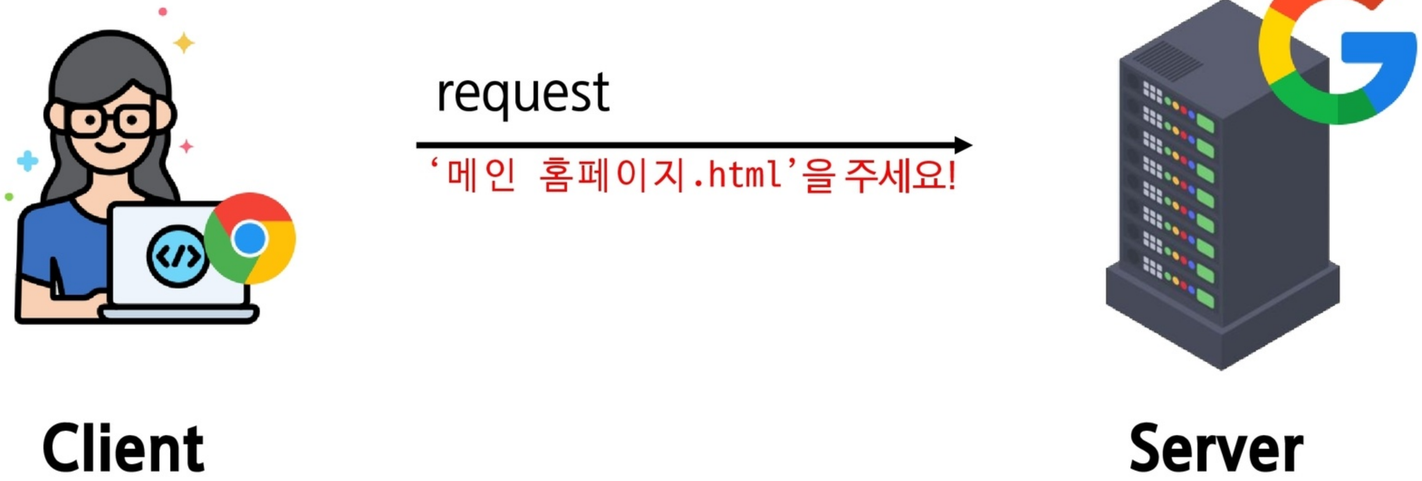
<그림2_Django_Intro & Design Pattern_웹 페이지를 보게 되는 과정 1>

## 우리가 웹 페이지를 보게 되는 과정 (2/4)

2. 웹 브라우저는 인터넷에 연결된 전세계 어딘 가에 있는
   구글 컴퓨터(서버)에게 '메인 홈페이지.html'파일을 달라고 요청

**HTML**
웹 페이지의 의미와 구조를 정의하는 언어

<개념1_Django_Intro & Design Pattern_HTML>



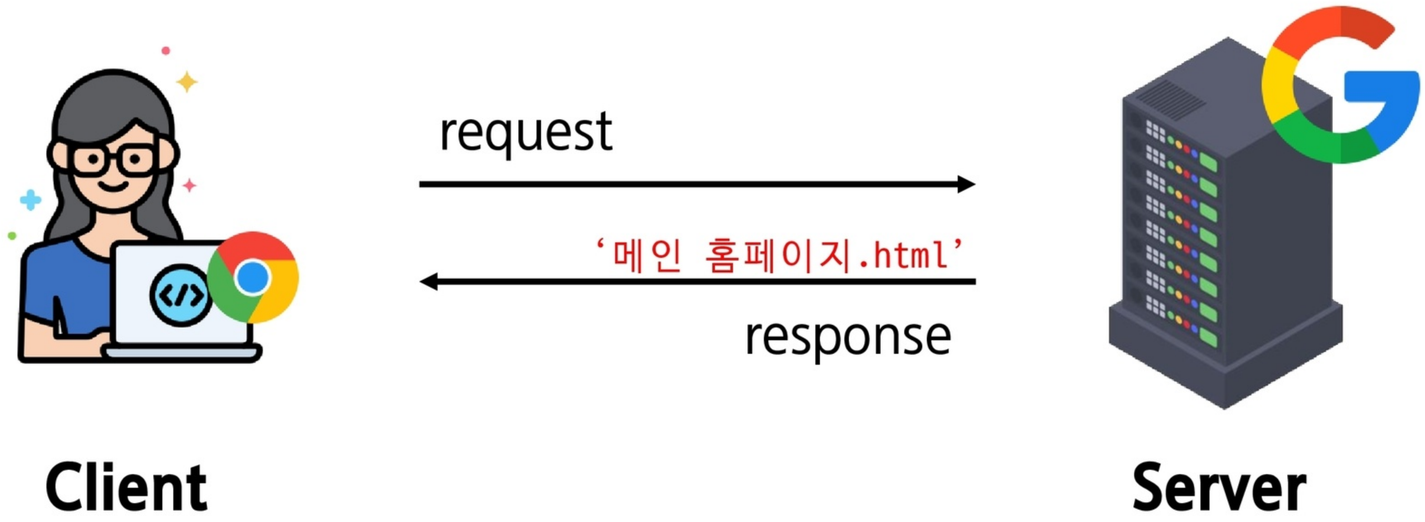

<그림3_Django_Intro & Design Pattern_웹 페이지를 보게 되는 과정 2>

## 우리가 웹 페이지를 보게 되는 과정 (3/4)

3. 요청을 받은 구글 컴퓨터는 데이터베이스에서
'메인 홈페이지.html'파일을 찾아 응답

**HTML**
웹 페이지의 의미와 구조를 정의하는 언어

<개념1_Django_Intro & Design Pattern_HTML>



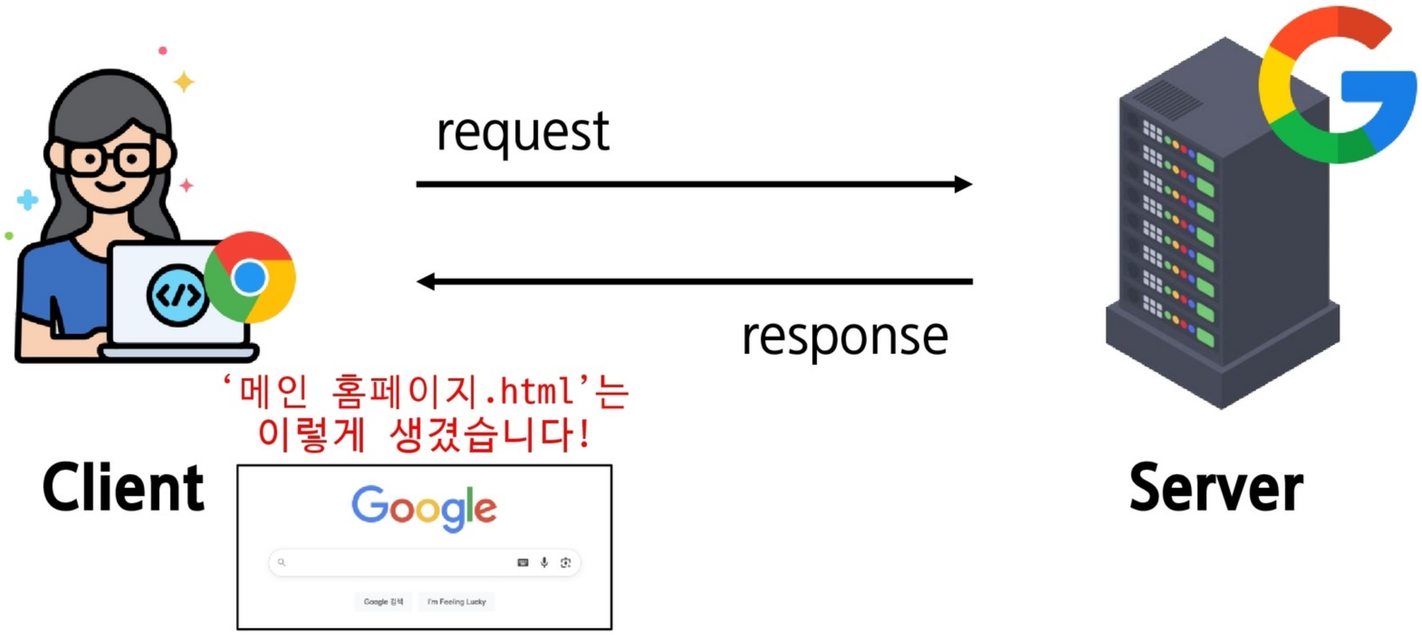

<그림4_Django_Intro & Design Pattern_웹 페이지를 보게 되는 과정 3>

## 우리가 웹 페이지를 보게 되는 과정 (4/4)

4. 웹 브라우저는 전달받은 '메인 홈페이지.html' 파일을
   사람이 볼 수 있도록 해석해주고 사용자는 구글의 메인 페이지를 보게 됨

**HTML**
웹 페이지의 의미와 구조를 정의하는 언어

<개념1_Django_Intro & Design Pattern_HTML>




<그림5_Django_Intro & Design Pattern_웹 페이지를 보게 되는 과정 4>

SAMSUNG SW AI ACADEMY FOR YOUTH

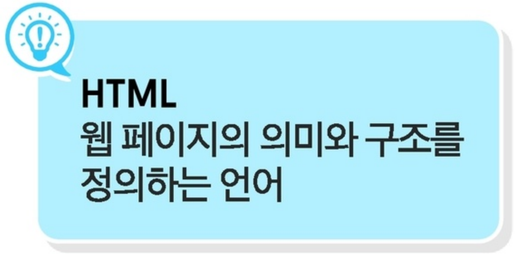
함께가요 미래로!
Enabling People

# Frontend & Backend

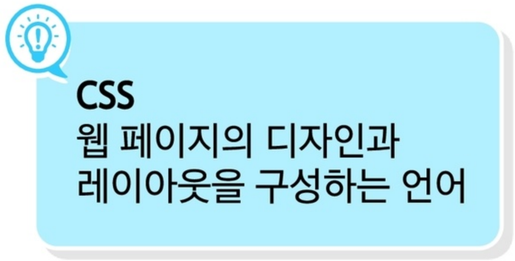
## 웹 개발에서의 Frontend와 Backend

- Frontend (프론트엔드)
  - 사용자 인터페이스(UI)를 구성하고, 사용자가 애플리케이션과 상호작용할 수 있도록 함

  ➢ HTML, CSS, JavaScript, 프론트엔드 프레임워크 등




<개념1_Django_Intro & Design Pattern_HTML>



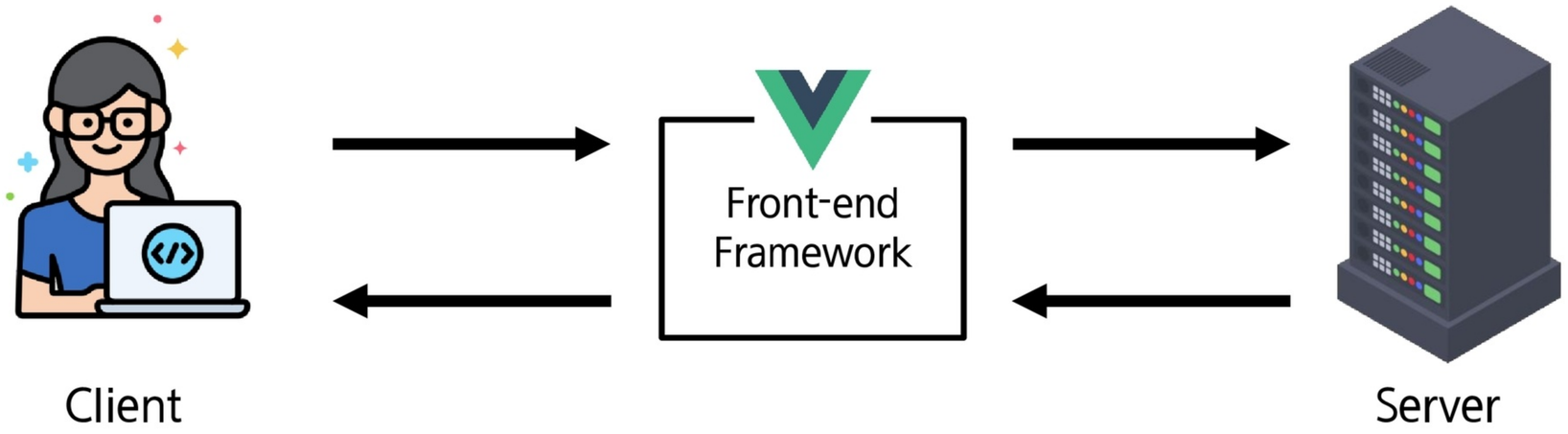

<개념2_Django_Intro & Design Pattern_CSS>

- Backend (백엔드)
  - 서버 측에서 동작하며, 클라이언트의 요청에 대한 처리와 데이터베이스와의 상호작용 등을 담당

  ➢ 서버 언어(Python, Java 등) 및 백엔드 프레임워크, 데이터베이스, API, 보안 등

## Frontend & Backend (1/2)

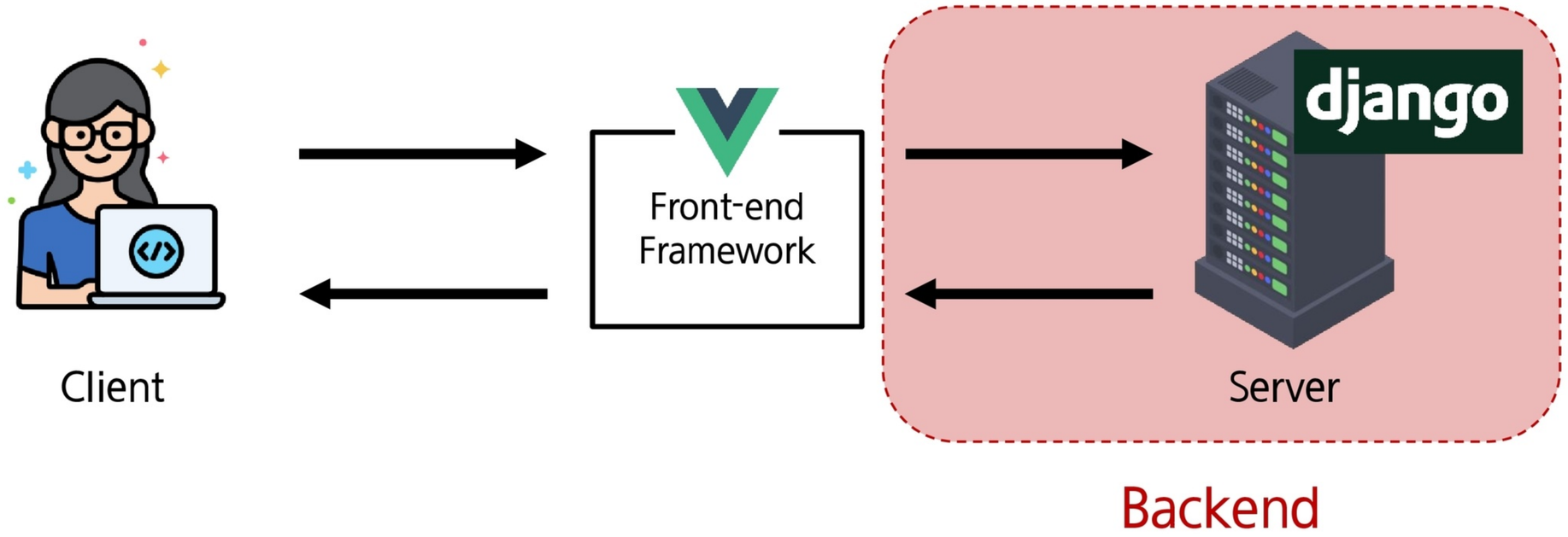
- Frontend: HTML, CSS, JavaScript를 이용해 UI를 구성
  - Vue.js: 프론트엔드로 활용되는 대표적인 프레임워크




<그림6_Django_Intro & Design Pattern_Frontend Backend 1>

## Frontend & Backend (2/2)

- Backend: 클라이언트의 요청에 대한 처리와 데이터베이스 상호작용
  - Django는 백엔드로 활용되는 대표적인 프레임워크




<그림7_Django_Intro & Design Pattern_Frontend Backend 2>

# Framework

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# Web Framework

## ‘웹 서비스 개발’에는 무엇이 필요할까?

# Web Framework

웹 애플리케이션을 빠르게
개발할 수 있도록 도와주는 도구

개발에 필요한 기본 구조, 규칙, 라이브러리 등을 제공
(로그인/로그아웃, 회원관리, 데이터베이스, 보안 등)

일반적인 ‘웹 서비스’에 필요한 다양한 보편적인 기능들이 존재합니다.
이런 기능을 전부 혼자 개발하기 보다는 이미 만들어진 도구를 효과적으로 활용하는 능력은 현대 웹 개발의 핵심이라고 볼 수 있습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# Django Framework

## Django Framework

<그림8_Django_Intro & Design Pattern_django logo>

Python 기반의 대표적인 웹 프레임워크

클라이언트 - 서버 구조의 서버를 구현하는 것이
Django를 배우는 목적입니다!

## 왜 Django를 사용할까?

- **다양성**
  - Python 기반으로 웹, 모바일 앱 백엔드, API 서버 및 빅데이터 관리 등 광범위한 서비스 개발에 적합
- **확장성**
  - 대량의 데이터에 대해 빠르고 유연하게 확장할 수 있는 기능을 제공
- **보안**
  - 취약점으로부터 보호하는 보안 기능이 기본적으로 내장되어 있음
- **커뮤니티 지원**
  - 개발자를 위한 지원, 문서 및 업데이트를 제공하는 활성화 된 커뮤니티

## 검증된 웹 프레임워크

- 대규모 트래픽 서비스에서도 안정적인 서비스 제공

<그림9_Django_Intro & Design Pattern_django 사용처>

https://djangostars.com/blog/10-popular-sites-made-on-django/
https://www.planeks.net/companies-that-use-django/
https://trio.dev/companies-using-django-applications/

## 웹 개발 시장을 주도하는 백엔드 프레임워크 (2024 ~ 2025)

1. Django (Python)
2. Spring Boot (Java)
3. ASP.NET Core (C#)
4. Laravel (PHP)
5. Express.js (Node.js)

https://survey.stackoverflow.co/2025/
https://www.jetbrains.com/lp/devecosystem-2024/
LinkedIn, Github Contributors, OSS Insight 자료 종합

# 가상 환경

## 가상 환경

# 가상 환경

## Virtual Environment

하나의 컴퓨터 안에서 또 다른 '독립된' 파이썬 환경

같은 집 안에 방을 따로 만들어 두고, 필요한 물건들을
각 방에만 들여놓는 것과 비슷합니다.

각 방의 물건은 서로 간섭하지 않게 됩니다.

## 가상 환경이 필요한 시나리오 1

1. 한 개발자가 2개의 프로젝트(A와 B)를 진행해야 하는 상황
2. 프로젝트 A는 requests 패키지 버전 10이 필요
3. 프로젝트 B는 requests 패키지 버전 20이 필요
4. 하지만 파이썬 환경에서 패키지는 1개의 버전만 존재할 수 있음
5. A와 B의 프로젝트의 다른 패키지 버전 사용을 위한 독립적인 개발 환경이 필요

**패키지**
연관된 모듈들을 하나의
디렉토리에 모아놓은 것

<개념3_Django_Intro & Design Pattern_패키지>

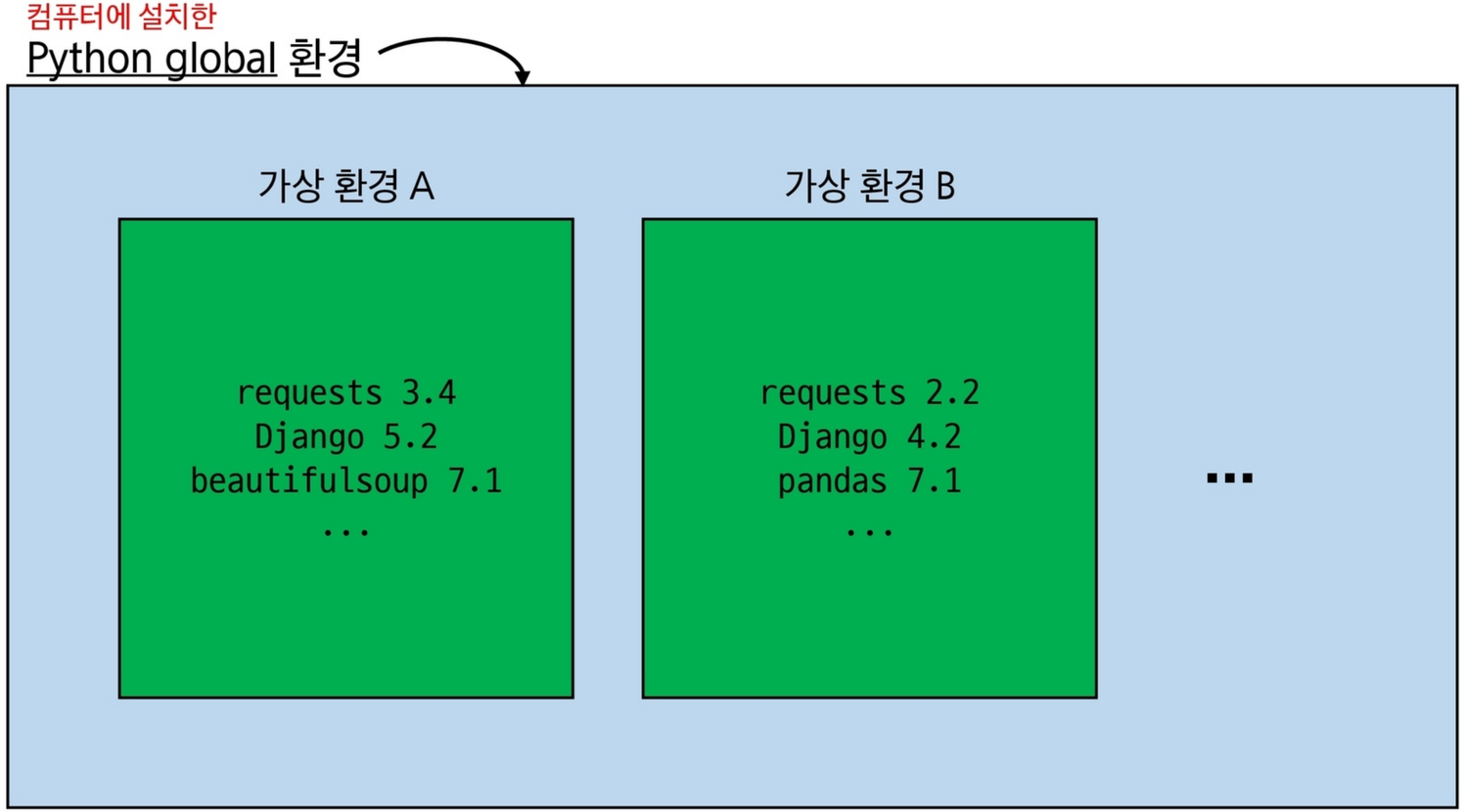
## 가상 환경이 필요한 시나리오 2

1. 한 개발자가 2개의 프로젝트(A와 B)를 진행해야 하는 상황
2. 프로젝트 A는 `water`라는 패키지가 필요
3. 프로젝트 B는 `fire`라는 패키지가 필요
4. 하지만 파이썬 환경에서 `water` 패키지와 `fire` 패키지를 함께 사용하면
   충돌이 발생하기 때문에 설치할 수 없다고 함
5. A와 B의 프로젝트의 패키지 충돌을 피하기 위해 각각 독립적인 개발 환경이 필요

**패키지**
연관된 모듈들을 하나의
디렉토리에 모아놓은 것

<개념3_Django_Intro & Design Pattern_패키지>

## Python 환경 구조 예시




<그림10_Django_Intro & Design Pattern_Python 환경 구조 예시>

**Python global**
컴퓨터 전체에서 사용되는
기본 Python 환경을 지칭

<개념4_Django_Intro & Design Pattern_Python global>

## 가상 환경 비유

- 같은 집(컴퓨터) 안에, 방(가상 환경)을 따로 만들어 두고,
- 필요한 물건(라이브러리, 패키지 등)을 그 방에만 들여놓는 것과 유사
- 방이 다르면 들여놓은 물건이 달라도 서로 간섭하지 않음

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 가상 환경 생성 및 활성화

## 가상 환경 생성 및 활성화 과정

1. 가상 환경 생성
2. 가상 환경 활성화
3. 가상 환경 종료

## 1. 가상 환경 생성

```
$ python -m venv venv
```

- 현재 디렉토리 안에 venv 라는 폴더가 생성됨
- venv 폴더 안에는 파이썬 실행 파일, 라이브러리 등을 담을 공간이 마련됨
- venv 라는 이름의 가상 환경을 생성한 것

※ 임의의 이름으로 생성이 가능하나 관례적으로 venv 이름을 사용

## 2. 가상 환경 활성화

```
$ source venv/Scripts/activate
```

- 활성화 후, 프롬프트 앞에 (venv)와 같이 표시된다면 성공한 것

### ※ Mac / Linux에서는 명령어가 다르니 주의

```
$ source venv/bin/activate
```

**TIP**

- 사용하고 있는 컴퓨터의 OS(Windows, Macos, Linux 등)에 따라 만들어진 venv 폴더의 구조가 다릅니다.
- 그렇기 때문에 가상 환경을 실행하는 명령어의 구조도 다릅니다!

## 3. 가상 환경 종료

```
$ deactivate
```

- 활성화한 상태에서 deactivate 명령을 입력하면, 다시 Python Global 환경으로 돌아옴

# 의존성 패키지

의존성

# 의존성

## Dependencies

하나의 소프트웨어가 동작하기 위해 필요로 하는
다른 소프트웨어나 라이브러리

컴퓨터가 잘 작동하기 위해서는 CPU, GPU, RAM 등 이 필요합니다.
이들은 컴퓨터의 입장에서는 의존성이라 볼 수 있습니다.

## 의존성 패키지

# 의존성 패키지

프로젝트가 의존하는 “개별 라이브러리”들을 가리키는 말

➢ “프로젝트가 실행되기 위해 꼭 필요한 각각의 패키지”

컴퓨터의 의존성인 CPU, GPU, RAM 등을 각각 의존성 패키지라고 할 수 있습니다.

## 패키지 목록 확인

```
$ pip list
```

- 현재 가상 환경에 설치된 라이브러리 목록을 확인하는 명령어
- 갓 생성된 가상 환경은 추가 설치된 패키지가 없음

**pip**
외부 패키지들을 설치하도록 도와주는 파이썬의 패키지 관리 시스템

<개념5_Django_Intro & Design Pattern_pip>

```
Package    Version
---------- -------
pip        24.0
setuptools 65.5.0
```

<그림11_Django_Intro & Design Pattern_가상환경 패키지 1>

※ 기본 환경 구성을 위한 pip, setuptools 정도만 존재합니다.
Python 버전에 따라 다를 수 있습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 의존성 관리: 환경의 일치

## 1. 의존성 기록 (내보내기)

```
$ pip freeze > requirements.txt
```

**pip freeze**
가상 환경에 설치된 모든 패키지를 버전과 함께 특정한 형식으로 출력하는 명령어

<개념7_Django_Intro & Design Pattern_pip freeze>

- 현재 가상환경에 설치된 모든 패키지와 버전을 파일로 저장

※ **다른 파일명으로도 가능하나 관례적으로 `requirements.txt`를 사용합니다.**

**TIP**

- “>”는 pip 명령어가 아닌 CLI(Shell)의 Redirection operator로, 이전 명령어의 출력을 파일로 redirect, 즉 생성하고 작성 합니다.
- 같은 명령어를 다시 사용할 경우 이전 파일의 내용을 덮어씁니다.

## 2. 의존성 설치 (가져오기)

```
$ pip install -r requirements.txt
```

- requirements.txt에 명시된 라이브러리를 한 번에 설치

**TIP**

- `-r` 옵션은 `--requirement`의 축약형 입니다.
- 패키지 이름을 하나씩 직접 입력하는 대신 "지정된 텍스트 파일(예: requirements.txt)을 읽어서 그 안에 적힌 모든 패키지를 설치해라"라고 지시하는 옵션입니다.

## 의존성 리스트 예시

- requests 패키지 설치 후 현재 가상 환경의 패키지 목록 변화
- 1개만 설치되는 것이 아님
  - requests가 필요로 하는 의존성들이 존재하기 때문

**requests**
Python에서 HTTP 요청을
보낼 수 있게 해주는 패키지

<개념6_Django_Intro & Design Pattern_requests>

```
Package    Version
---------- -------
pip        24.0
setuptools 65.5.0
```

```
Package                  Version
------------------------ ---------
certifi                  2025.6.15
charset-normalizer       3.4.2
idna                     3.10
pip                      24.0
requests                 2.32.4
setuptools               65.5.0
urllib3                  2.5.0
```

<그림12_Django_Intro & Design Pattern_가상환경 패키지 2>

## 의존성 리스트 공유 시나리오

1. 2명(A와 B)의 개발자가 하나의 프로젝트를 함께 개발한다고 가정
2. 개발자 A가 먼저 가상 환경을 생성 후 프로젝트를 설정하고 관련된 패키지를 설치하고 개발하다가 협업을 위해 GitHub에 프로젝트를 push
3. 개발자 B는 해당 프로젝트를 clone 받고 실행해보려 하지만 실행되지 않음
4. 개발자 A가 이 프로젝트를 위해 어떤 패키지를 설치했고, 어떤 버전을 설치했는지 A의 가상 환경 상황을 알 수 없음
5. 가상 환경에 대한 정보 즉 패키지 목록이 공유되어야 함

## 의존성 패키지 관리가 필요한 이유

- 환경의 통일
  - 버전이 다른 경우 함수명이나 동작이 달라질 수 있음
  - "내 컴퓨터에선 되는데 네 컴퓨터에선 안 돼" 문제 해결
- 협업 효율
  - 프로젝트가 커질수록 사용하는 패키지의 개수도 늘어나게 됨
  - 팀원 간 동일한 개발 환경을 즉시 구축 가능

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 가상 환경 주의사항

## 가상 환경 주의사항 및 권장사항

1. 가상 환경에 “들어가고 나오는” 것이 아니라 사용할 Python 환경을 “On/Off”로 전환하는 개념
   - 가상 환경 활성화는 현재 터미널 환경에만 영향을 끼침
   - 새 터미널 창을 열면 다시 활성화해야 함
2. 프로젝트마다 별도의 가상 환경을 사용
3. 일반적으로 가상 환경 폴더 venv는 관련된 프로젝트와 동일한 경로에 위치시킴
4. 폴더 venv는 `.gitignore`파일에 작성되어 원격 저장소에 공유하지 않음
   - 저장소 크기를 줄여 효율적인 협업과 배포를 가능하게 하고
   - OS 별 차이점으로 인한 문제를 방지하기 위함
   - 대신 `requirements.txt`를 공유하여 각자의 가상 환경을 구성

## 가상 환경이 필요한 이유

1. 프로젝트마다 다른 버전의 라이브러리 사용
   - 한 프로젝트에서는 Django 4.2를 다른 프로젝트에서는 Django 5.2를 사용해야 할 수도 있음
   - 가상 환경을 사용하면 서로 다른 버전을 동시에 설치해도 충돌 없이 각각의 프로젝트를 유지할 수 있음
2. 의존성 충돌 방지
   - 프로젝트별로 라이브러리를 독립적으로 관리하게 해 줌
   - 여러 프로젝트가 동시에 같은 라이브러리를 쓰더라도 버전 충돌 문제를 예방
3. 팀원 간 협업
   - 누구든지 동일한 방식으로 가상 환경을 만들어서, 똑같은 버전의 라이브러리를 설치하면 에러 가능성을 줄일 수 있음

## 요약

1. 가상 환경 생성(python –m venv venv)
2. 가상 환경 활성화 (source venv/Scripts/activate)
3. 필요한 의존성 패키지 설치 (pip install)
4. 현재 환경의 패키지 목록을 “pip freeze > requirements.txt”로 저장하여 의존성을 관리
5. 다른 컴퓨터나 팀원도 같은 환경이 필요하다면, “pip install –r requirements.txt”로 동일한 버전의 라이브러리를 설치
6. 작업이 끝나면 “deactivate”로 가상 환경을 비활성화

# Django 프로젝트

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# Django 프로젝트 생성 및 서버 실행

## Django 프로젝트 생성 및 서버 실행 과정

1. Django 설치
2. 프로젝트 생성
3. 서버 실행

## 1. Django 설치

```
$ pip install django
```

- 현재 환경에 Django 패키지를 설치

```
Downloading django-5.2.[illegible]-py3-none-any.whl (8.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.3/8.3 MB 50.1 MB/s eta 0:00:00
```

<그림13_Django_Intro & Design Pattern_Django 설치>

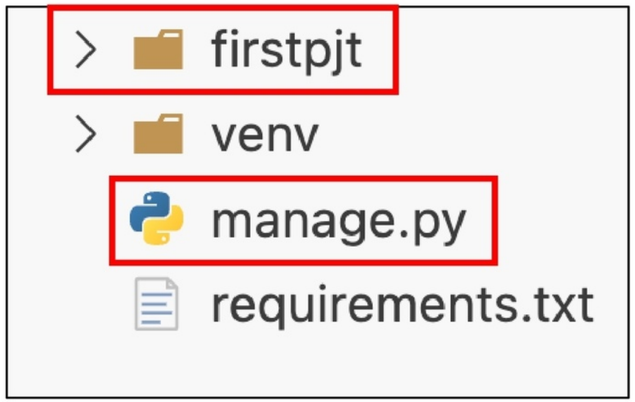
※ Django 버전을 명시하지 않을 경우 python 3.11 기준 최신 버전인 5.2.x 버전이 설치 됨

## 2. 프로젝트 생성

```
$ django-admin startproject firstpjt .
```

- “firstpjt”라는 이름의 django 프로젝트를 생성

추가되는 파일 및 폴더




<그림14_Django_Intro & Design Pattern_Django 프로젝트 생성>

## 3. 서버 실행

```
$ python manage.py runserver
```

- “manage.py”와 동일한 위치에서 명령어 진행

```
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you
apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
[redacted]
Django version 5.2.4, using settings 'firstpjt.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Use
 a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en
/5.2/howto/deployment/
```

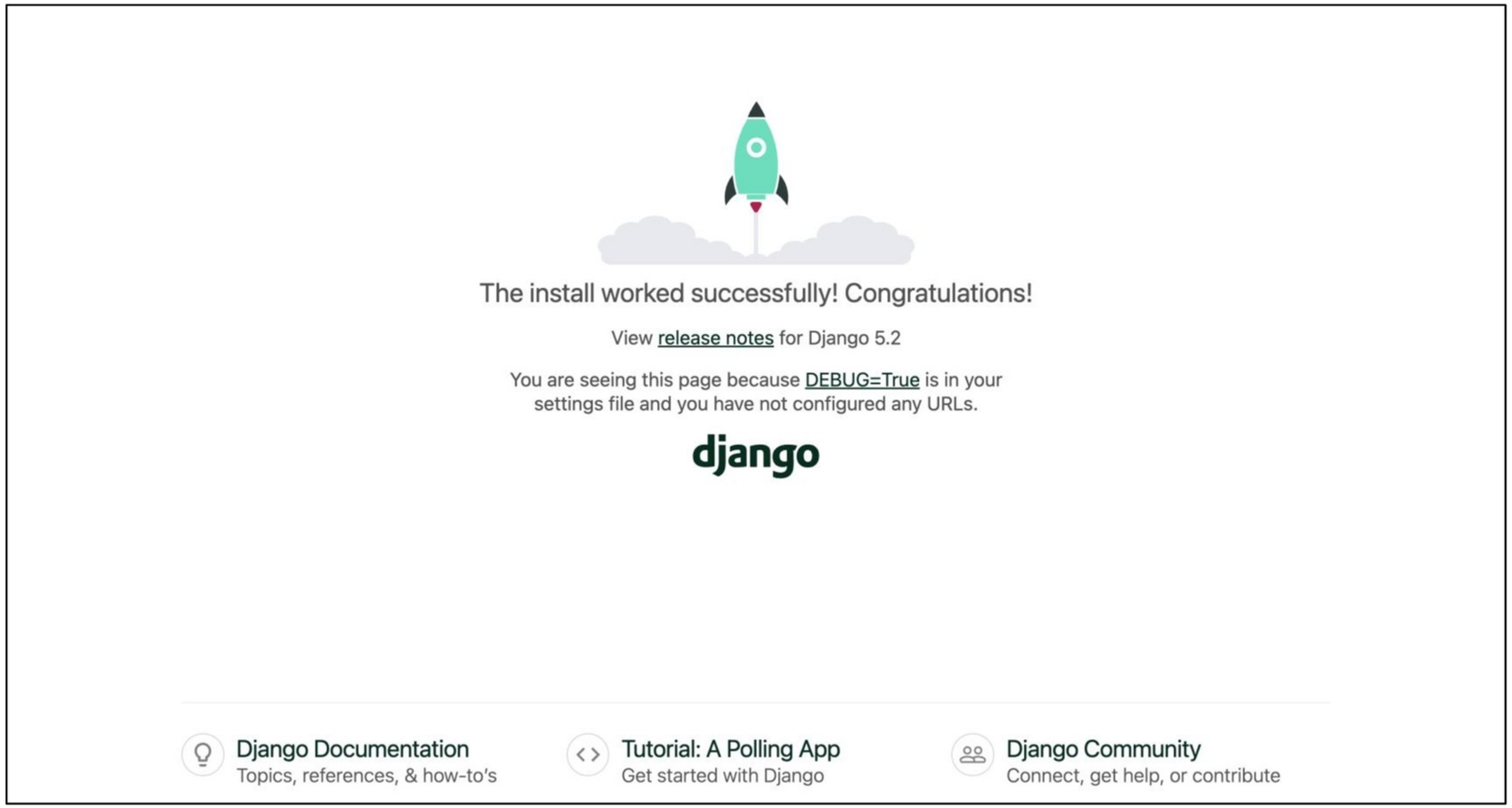
※ 이 경고는 추후 챕터에서 해결

※ 이 경고는 개발 단계와는 무관한 경고로,
신경 쓰지 않아도 괜찮음

<그림15_Django_Intro & Design Pattern_Django 실행>

## 서버 확인

- http://127.0.0.1:8000/ 접속 후 확인




<그림16_Django_Intro & Design Pattern_Django 실행 결과 화면>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# Django Design Pattern

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# Design Pattern

## 디자인 패턴

# 디자인 패턴

## Design Pattern

소프트웨어 설계에서 반복적으로 발생하는 문제에 대한,
검증되고 재사용 가능한 일반적인 해결책

"애플리케이션의 구조는 이렇게 구성하자"라는 모범 답안 또는 관행
대표적인 디자인 패턴: MVC

## MVC 디자인 패턴

- 하나의 애플리케이션을 구조화하는 대표적인 구조적 디자인 패턴
- Model
  - 데이터 및 비즈니스 로직을 처리
- View
  - 사용자에게 보이는 화면을 담당
- Controller
  - 사용자의 입력을 받아 Model과 View를 제어

➢시각적 요소와 뒤에서 실행되는 로직을 서로 영향 없이, 독립적이고 쉽게 유지 보수할 수 있는 애플리케이션을 만들기 위함

## MTV 디자인 패턴

# MTV 디자인 패턴

## (Model, Template, View)

Django에서 애플리케이션을 구조화하는 디자인 패턴

View ⟶ Template

Controller ⟶ View

※ 기존 MVC 패턴과 동일하나 단순히 명칭을 다르게 정의한 것

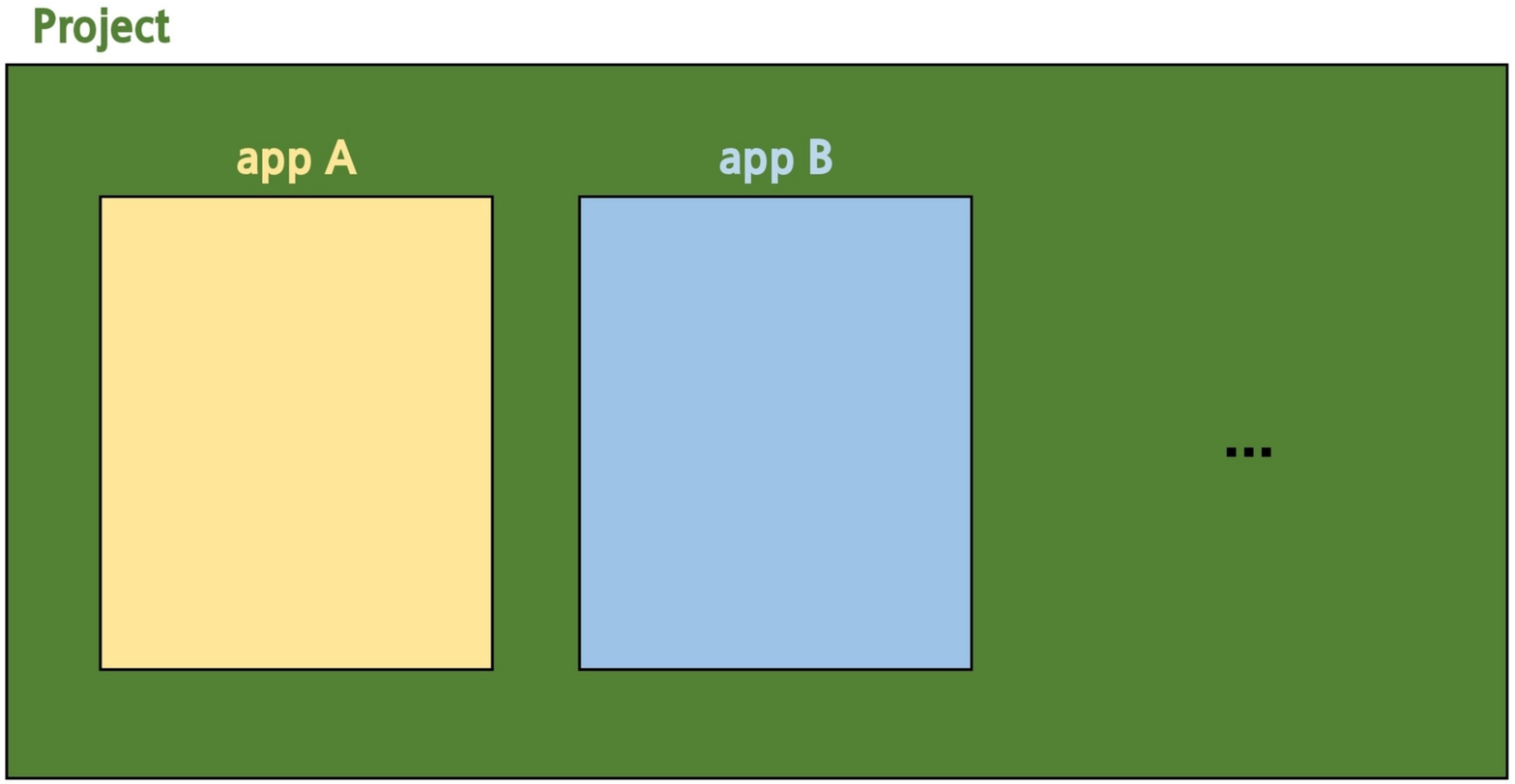
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 프로젝트와 앱

## 프로젝트와 앱

- Django에서 프로젝트와 앱의 관계



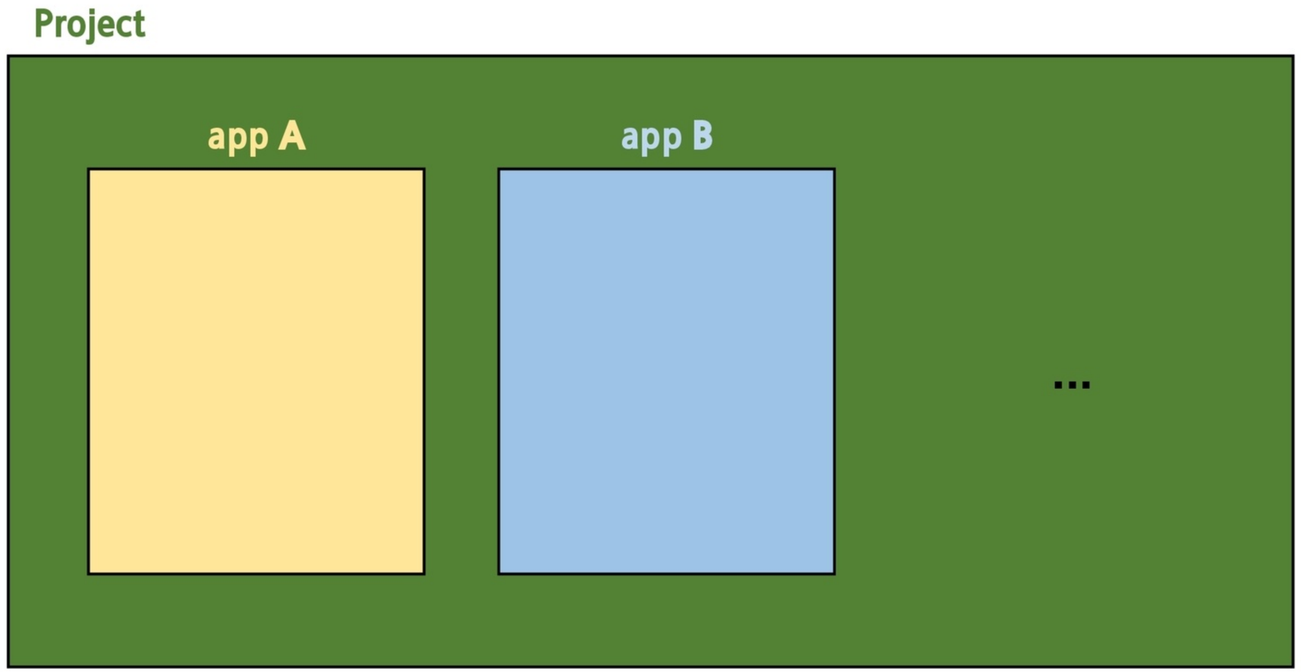

<그림17_Django_Intro & Design Pattern_프로젝트와 앱>

## Django project

- 애플리케이션의 집합
  - DB 설정, URL 연결, 전체 앱 설정 등을 처리

## Django application

- 독립적으로 작동하는 기능 단위 모듈
  - 각자 특정한 기능을 담당
  - 다른 앱들과 함께 하나의 프로젝트를 구성




<그림17_Django_Intro & Design Pattern_프로젝트와 앱>

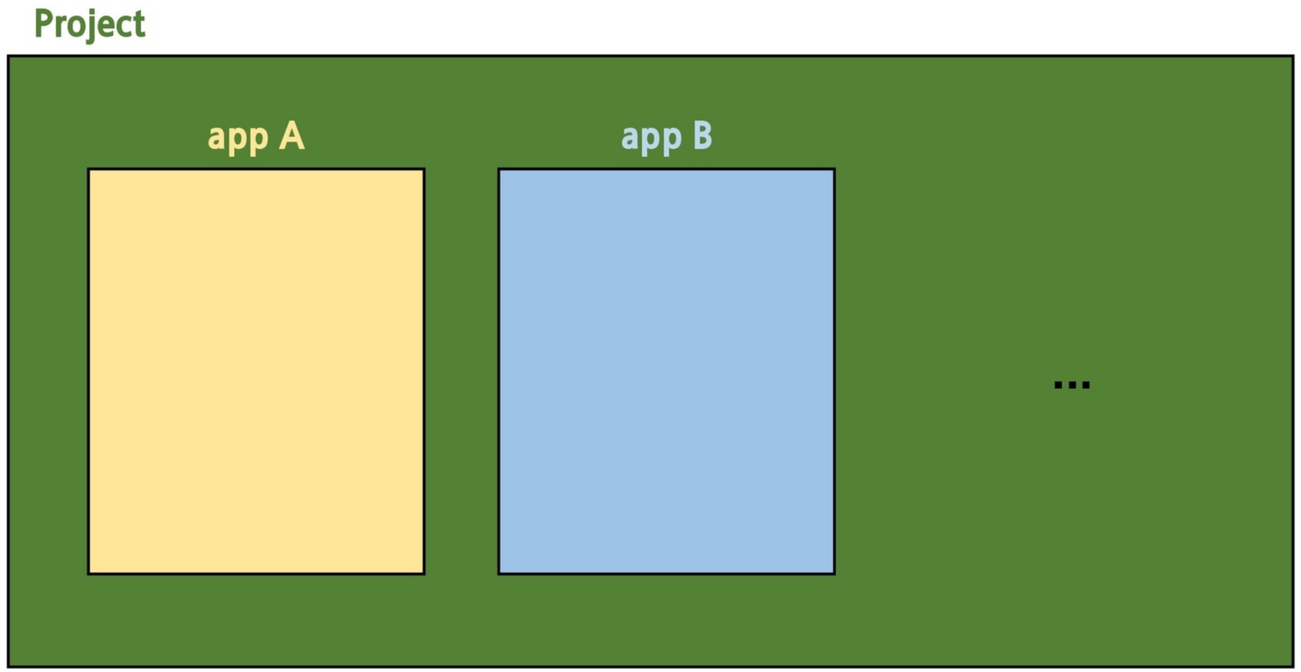
## 만약 온라인 커뮤니티 카페를 만든다면?

### 프로젝트

카페
(전체 설정 담당)

### 앱

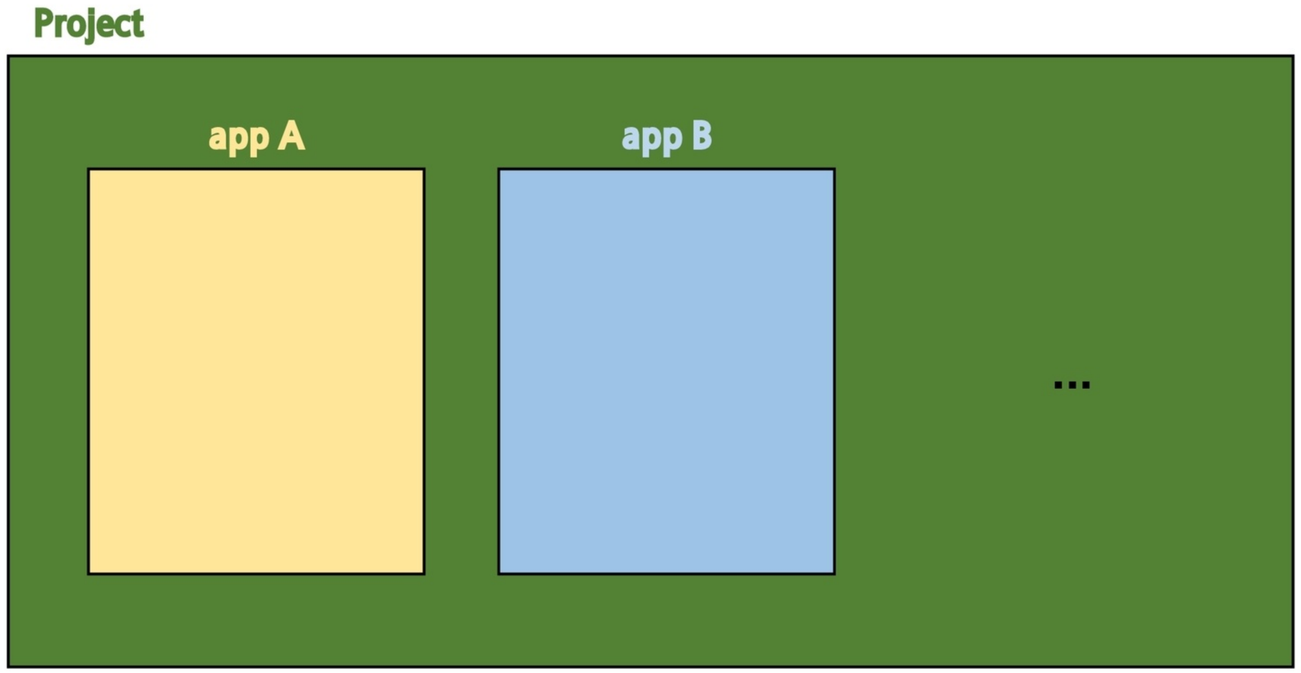
게시글, 댓글, 회원 관리 등
(DB, 인증, 화면)




<그림17_Django_Intro & Design Pattern_프로젝트와 앱>

## 다양한 서비스에 대입해보면

- **카페 프로젝트**
  - 게시글, 댓글, 회원 관리 앱
- **쇼핑몰 프로젝트**
  - 상품 조회, 배송 조회, 결제 앱
- **교육 관리 프로젝트**
  - 학생 관리, 문제 관리, 점수 관리 앱



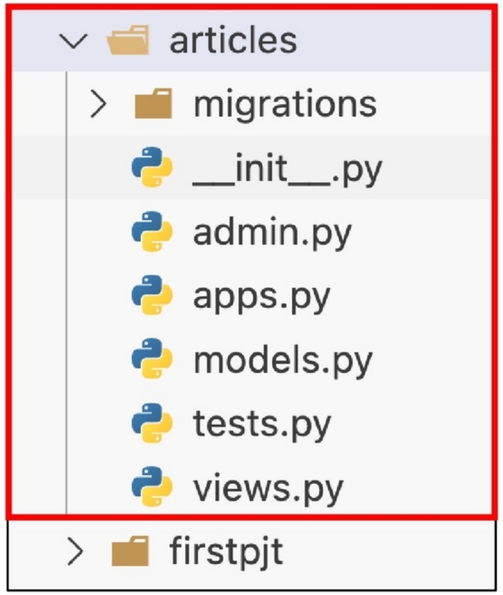

<그림17_Django_Intro & Design Pattern_프로젝트와 앱>

## 1. 앱 생성

```
$ python manage.py startapp articles
```

- ‘articles’라는 폴더와 내부에 여러 파일이 새로 생성됨

※ 앱의 이름은 ‘복수형’으로 지정하는 것을 권장




<그림18_Django_Intro & Design Pattern_앱 생성>

## 2. 앱 등록

- 반드시 앱을 생성(1)한 후에 등록(2)해야 함 ※ 등록 후 생성은 불가

```python
# settings.py

INSTALLED_APPS = [  33번 째 라인
    'articles',
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

※ 등록을 먼저 할 경우, 생성을 위한 명령어 실행 중
아직 존재하지 않는 articles 앱을 찾으려다 실패하기 때문

SAMSUNG SW AI ACADEMY FOR YOUTH

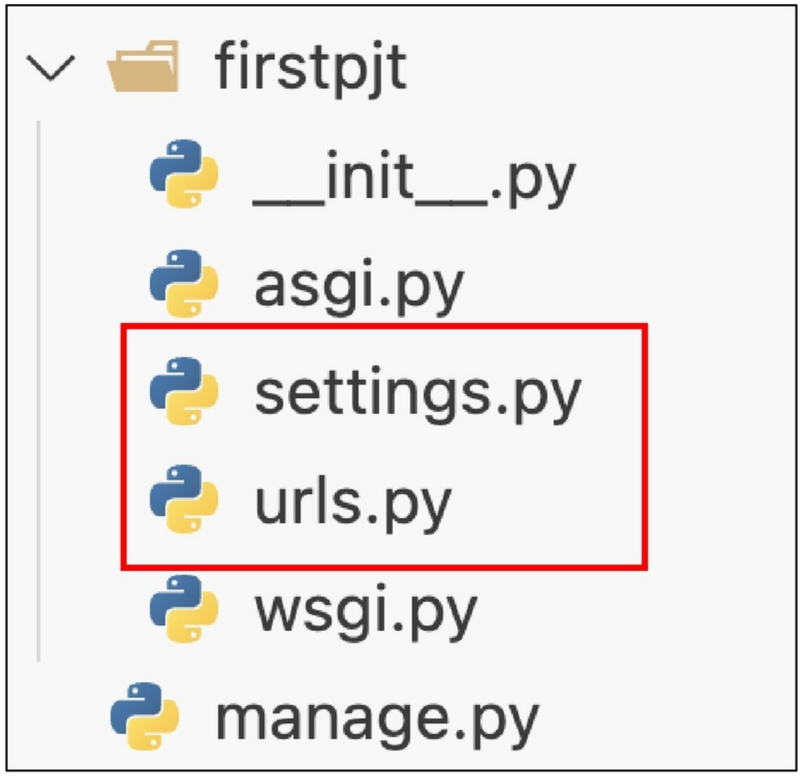
함께가요 미래로!
Enabling People

# 프로젝트 및 앱 구조

## 프로젝트 구조 (1/2)

- `settings.py`
  - 프로젝트의 모든 설정을 관리

- `urls.py`
  - 요청 들어오는 URL에 따라 이에 해당하는 적절한 views를 연결

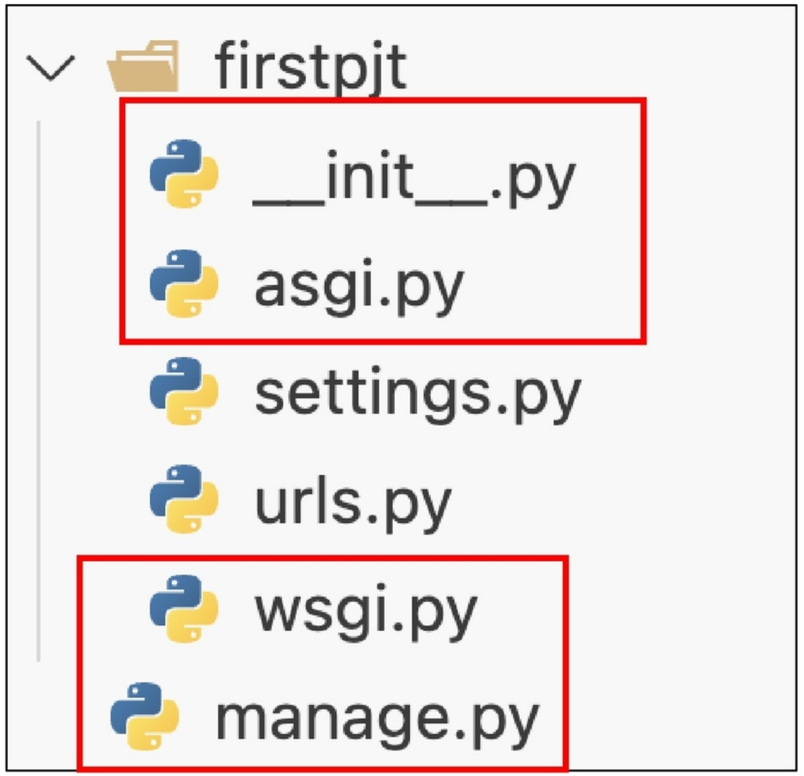



<그림19_Django_Intro & Design Pattern_프로젝트 구조 1>

## 프로젝트 구조 (2/2)

- `__init__.py`
  - 해당 폴더를 패키지로 인식하도록 설정하는 파일
- `asgi.py`
  - 비동기식 웹 서버와의 연결 관련 설정
- `wsgi.py`
  - 웹 서버와의 연결 관련 설정
- `manage.py`
  - Django 프로젝트와 다양한 방법으로 상호작용하는 커맨드라인 유틸리티

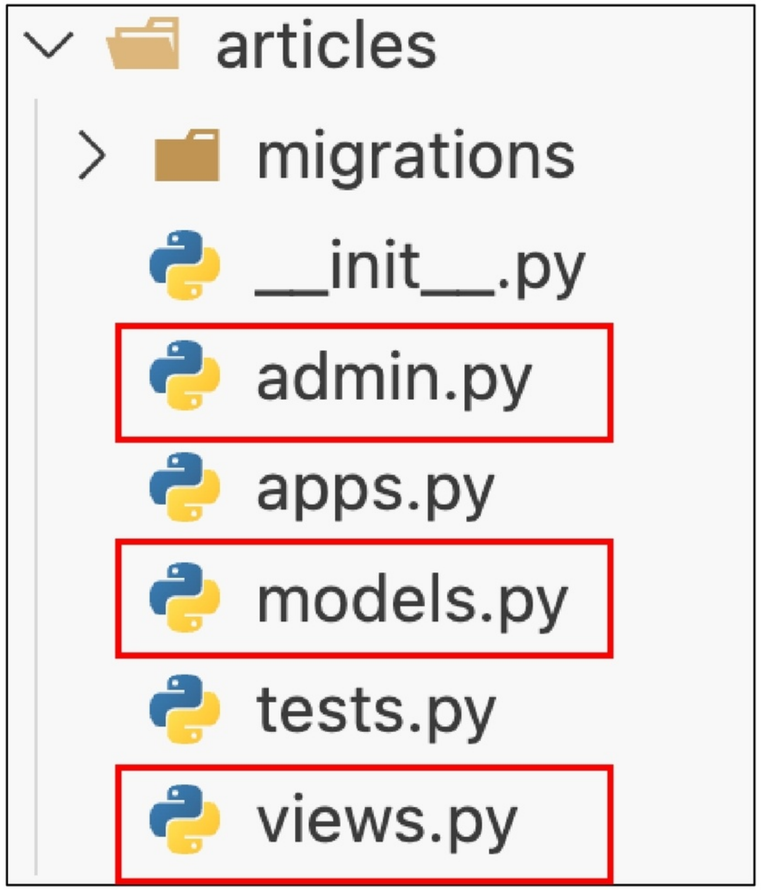
※ 수업 과정에서 수정할 일 없음




<그림20_Django_Intro & Design Pattern_프로젝트 구조 2>

## 앱 구조 (1/2)

- `admin.py`
  - 관리자용 페이지 설정
- `models.py`
  - DB와 관련된 Model을 정의
  - MTV 패턴의 M
- `views.py`
  - HTTP 요청을 처리하고 해당 요청에 대한 응답을 반환
    - url, model, template과 연동
  - MTV 패턴의 V



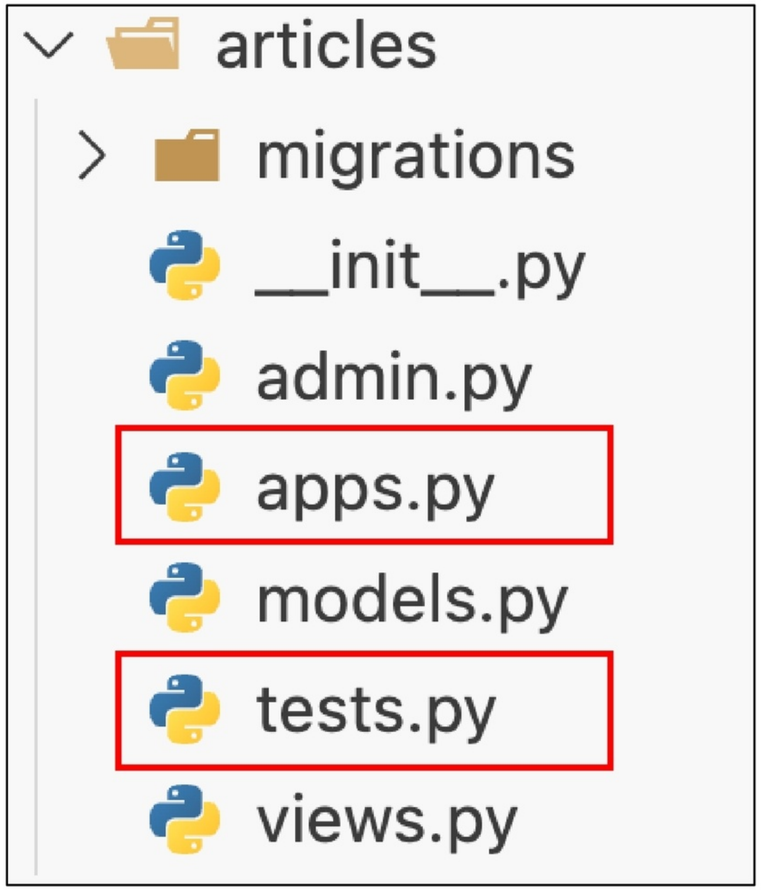

<그림21_Django_Intro & Design Pattern_앱 구조 1>

## 앱 구조 (2/2)

- apps.py
  - 앱의 정보가 작성된 곳

- tests.py
  - 프로젝트 테스트 코드를 작성하는 곳

※ 수업 과정에서 수정할 일 없음

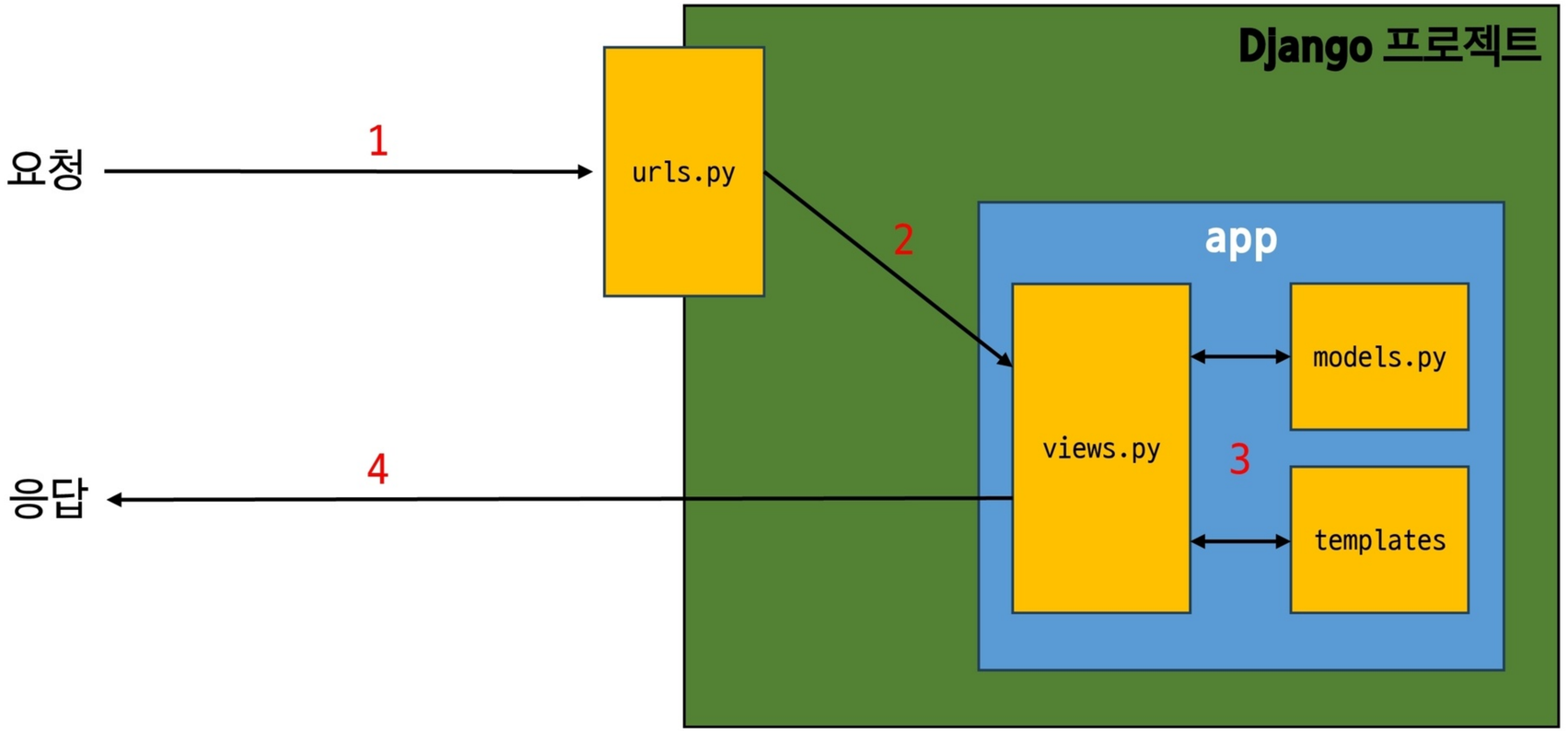



<그림22_Django_Intro & Design Pattern_앱 구조 2>

# 요청과 응답

## Django와 요청 & 응답 (1/2)

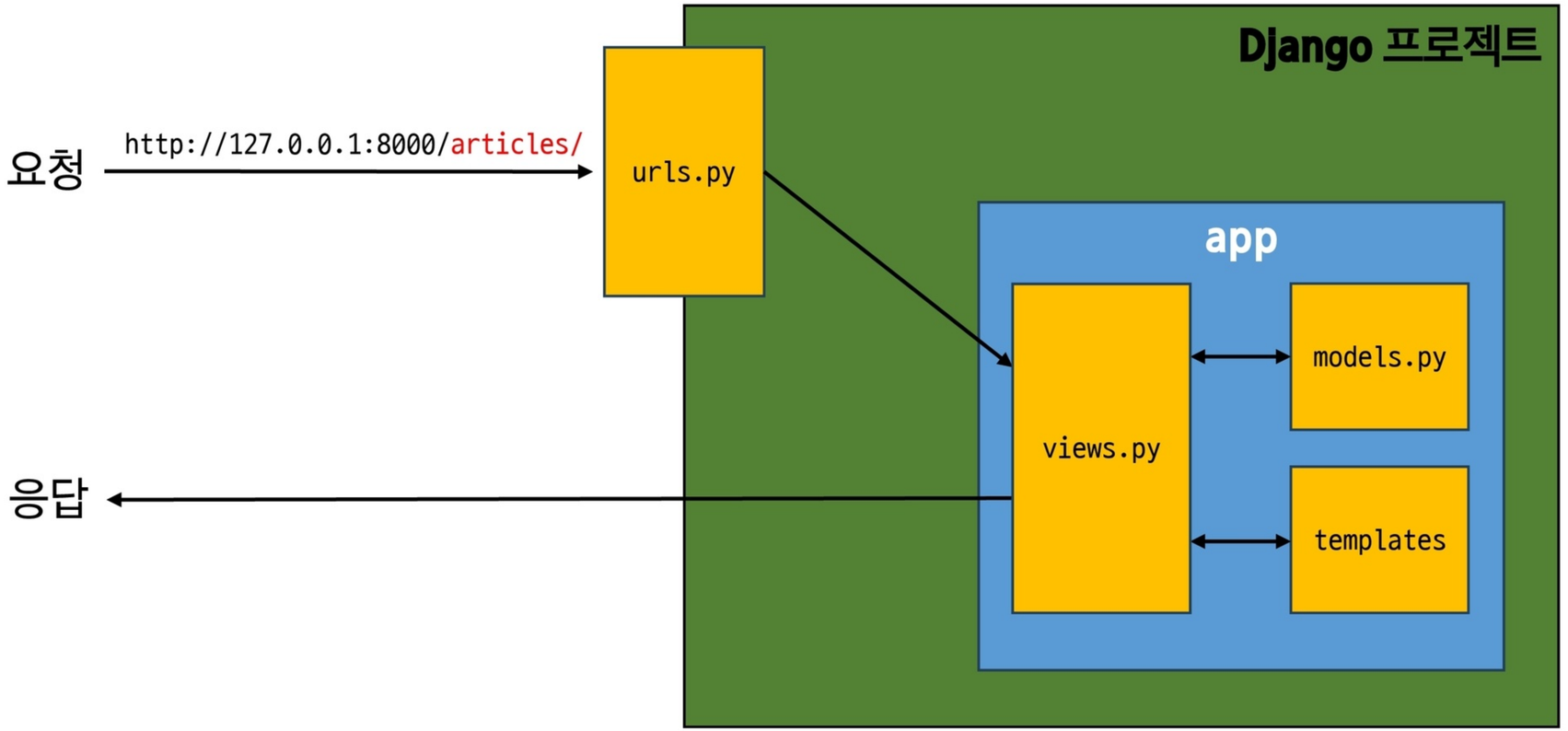
- 사용자가 서버에 접속하는 과정이다.




<그림23_Django_Intro & Design Pattern_Django 요청 응답 1>

## Django와 요청 & 응답 (2/2)

- 사용자가 서버에 접속하는 과정이다.




<그림24_Django_Intro & Design Pattern_Django 요청 응답 2>

## 1. URLs

- http://127.0.0.1:8000/articles/ 로 요청이 왔을 때
  - request 객체를 **views** 모듈의 index view 함수에 전달하며 호출

```python
# urls.py

from django.contrib import admin
from django.urls import path
from articles import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('articles/', views.index),
]
```

※ articles 패키지에서 views 모듈을 가져오는 것

※ url 경로는 반드시 '/'(slash)로 끝나야 함

## 2. View

- view 함수가 정의되는 곳
  - 특정 경로에 있는 template과 request 객체를 결합해 응답 객체를 반환

```python
# views.py

from django.shortcuts import render


def index(request):
    return render(request, 'articles/index.html')
```

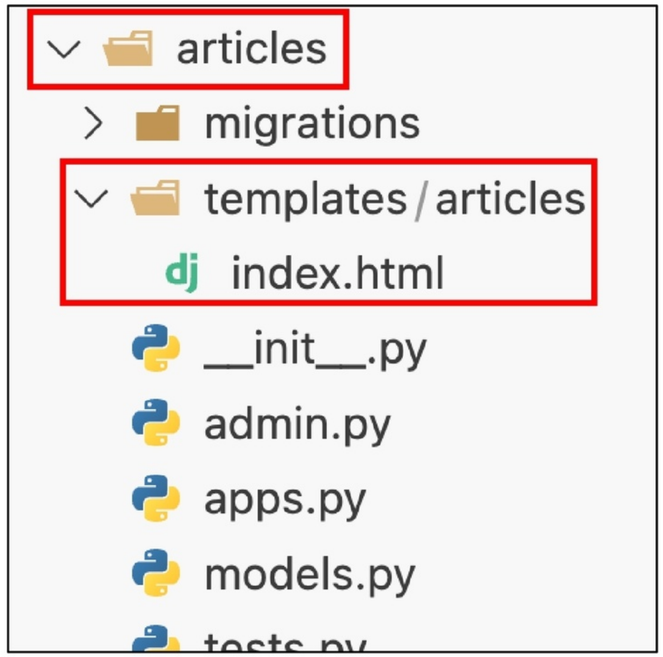
※ 모든 view 함수는 첫번째 인자로 요청 객체를 필수적으로 받음

※ 매개변수 이름은 request가 아니어도 되지만 관례적으로 request로 작성함

## 3. Template

1. articles 앱 폴더 안에 templates 폴더 생성
2. templates 폴더 안에 articles 폴더 생성
3. articles 폴더 안에 템플릿 파일 생성

※ 폴더명은 반드시 templates 여야 하며 개발자가 직접 생성해야 함




```
<!-- articles/index.html -->

<!DOCTYPE html>
<html lang="en">
<head>
  …
  <title>Document</title>
</head>
<body>
  <h1>Hello, Django!</h1>
</body>
</html>
```

<그림25_Django_Intro & Design Pattern_템플릿 위치>

## Django에서 template을 인식하는 경로 규칙

app폴더 / templates / articles / index.html

app폴더 / templates / example.html

- Django는 이 지점까지 기본 경로로 인식
  - view 함수에서 template 경로 작성 시 해당 지점 이후의 경로를 작성해야 함

```python
# views.py

from django.shortcuts import render


def index(request):
    return render(request, 'articles/index.html')
```

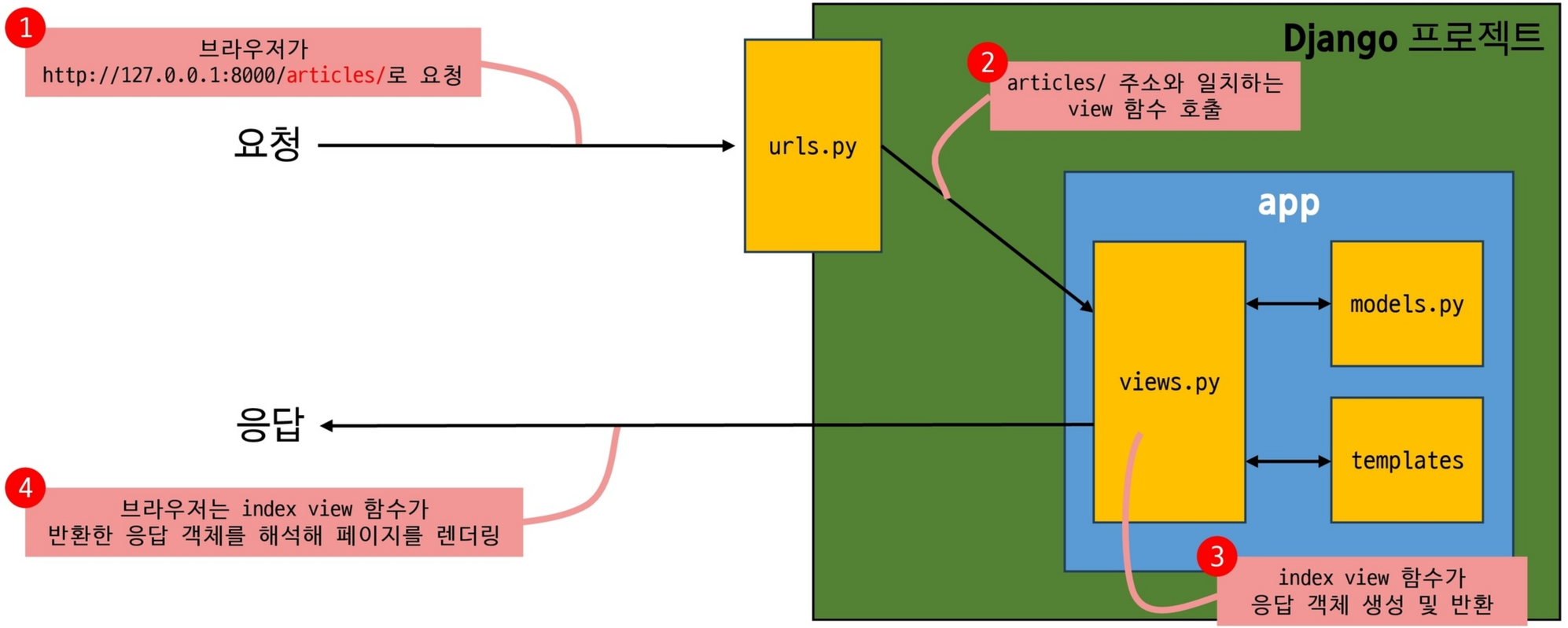
## 요청 후 응답 페이지 확인

- 브라우저로 요청을 해보자
  - 주소창에 http://127.0.0.1:8000/articles/ 입력

<그림26_Django_Intro & Design Pattern_응답 페이지>

## 요청과 응답 과정 정리

- 그림을 다시 보자




<그림27_Django_Intro & Design Pattern_Django 요청 응답 3>

## 데이터 흐름에 따른 코드 작성하기

- 사용자의 요청에서부터 데이터의 흐름은 URLs → View → Template
- 코드의 작성도 마찬가지로 데이터의 흐름을 따라 작성할 것

URLs
```
path('articles'/, view.index),
```

View
```
def index(request):
    return render(request, 'articles/index.html')
```

Template
```
articles/templates/articles/index.html
```

# 참고

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 가상 환경 생성 루틴

## Django 프로젝트 생성 전 루틴

1. 가상 환경 생성
2. 가상 환경 활성화
3. Django 설치
4. 패키지 목록 파일 생성
   - 패키지 설치시마다 진행

※ Python 3.9 이하일 경우 Django 4 버전이 설치되니 주의

```
# 1. 가상 환경(venv) 생성
$ python -m venv venv

# 2. 가상 환경 활성화
$ source venv/Scripts/activate

# 3. Django 설치
$ pip install django

# 4. 패키지 목록 파일 생성
$ pip freeze > requirements.txt
```

## Django 프로젝트를 git 저장소로 만드는 경우

1. 가상 환경 생성
2. 가상 환경 활성화
3. Django 설치
4. 패키지 목록 파일 생성
   - 패키지 설치시마다 진행
5. .gitignore 파일 생성
   - 첫 add 전 진행
6. git 저장소 생성
   - git init

   추가
7. Django 프로젝트 생성

venv가 제외되도록 할 것

※ db.sqlite3도 제외 됨

<그림28_Django_Intro & Design Pattern_git 저장소 추가>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

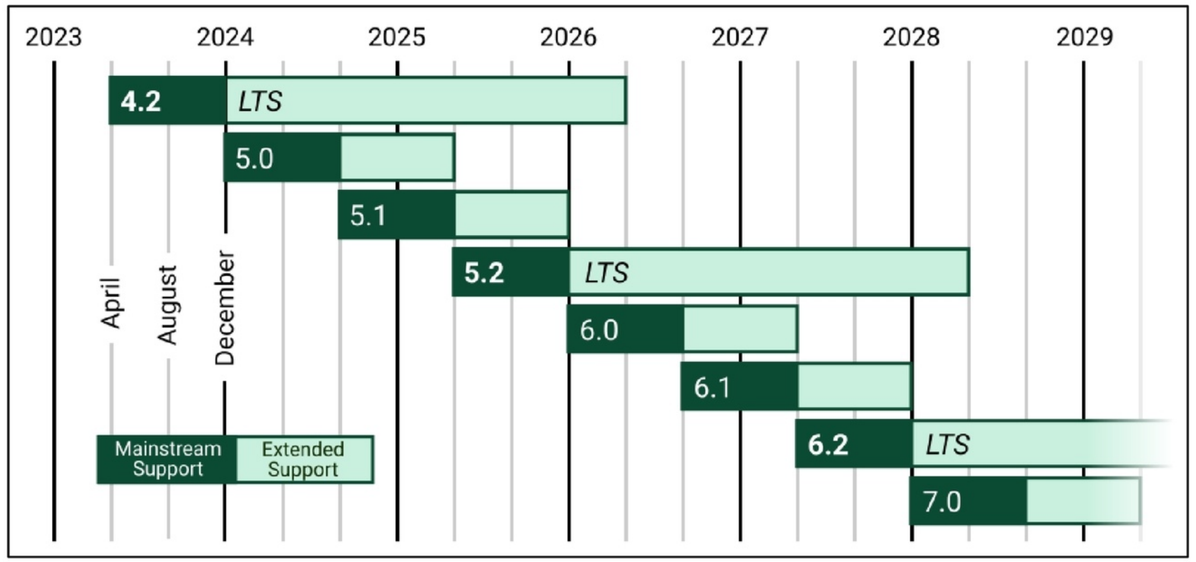
# Django 관련

## LTS (Long-Term Support)

- 프레임워크나 라이브러리 등의 소프트웨어에서 장기간 지원되는 안정적인 버전을 부를 때 사용
- 기업이나 대규모 프로젝트에서는 소프트웨어 업그레이드에 많은 비용과 시간이 필요하기 때문에 안정적이고 장기간 지원되는 버전이 필요

# Django Future Roadmap

- Django 버전 별 기술 지원 기간




| Django version | Python versions |
|---|---|
| 4.2 | 3.8, 3.9, 3.10, 3.11, 3.12 (added in 4.2.8) |
| 5.0 | 3.10, 3.11, 3.12 |
| 5.1 | 3.10, 3.11, 3.12, 3.13 (added in 5.1.3) |
| 5.2 | 3.10, 3.11, 3.12, 3.13 |

<그림29_Django_Intro & Design Pattern_Django future roadmap>

https://www.djangoproject.com/download/
https://docs.djangoproject.com/en/5.2/faq/install/

## Django는 Full Stack framework 인가요?

- 네

- 하지만 Django가 제공하는 Frontend 기능은 다른 전문적인 Frontend Framework 들에 비해서는 매우 미흡함
- 엄밀히 하자면 Full Stack 영역에서 Backend에 속한다고 볼 수 있음
- Full Stack 혹은 Backend Framework라 부름

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# Python 패키지 설치법

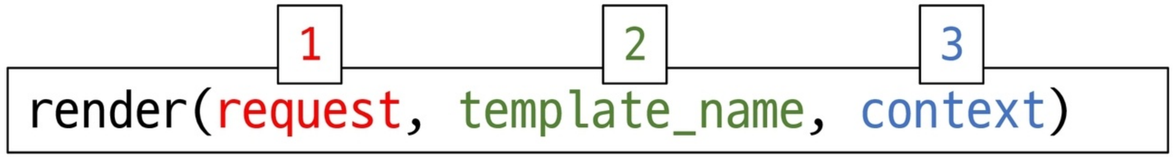
## Python 패키지 설치 시 버전 지정 방법

| 명령어 예시 | 기호 | 의미 |
|---|---|---|
| `pip install SomePackage` | (없음) | 최신 버전 설치 (버전 지정 없음) |
| `pip install SomePackage==1.0.5` | == | 특정 버전 정확히 설치 |
| `pip install SomePackage>=1.0.4` | >= | 최소 버전(1.0.4) 이상을 설치 (새 버전도 허용) |
| `pip install SomePackage~=1.0.4` | ~= | 호환 버전 이상(1.0.4 이상)<br>다음 마이너 버전 미만(예: <1.1.0)으로 설치<br>(마이너 업데이트 허용, 메이저 업데이트는 제한) |

<표1_Django_Intro & Design Pattern_Python 패키지 설치 버전 지정 방법>

# render 함수

## render 함수




- 주어진 템플릿을 주어진 컨텍스트 데이터와 결합하고 렌더링 된 텍스트와 함께 HttpResponse 응답 객체를 반환하는 함수

1. request
   - 응답을 생성하는 데 사용되는 요청 객체
2. template_name
   - 템플릿 이름의 경로
3. context ※ 추후 챕터에서 활용
   - 템플릿에서 사용할 데이터 (생략 가능)
   - 딕셔너리 타입으로 작성

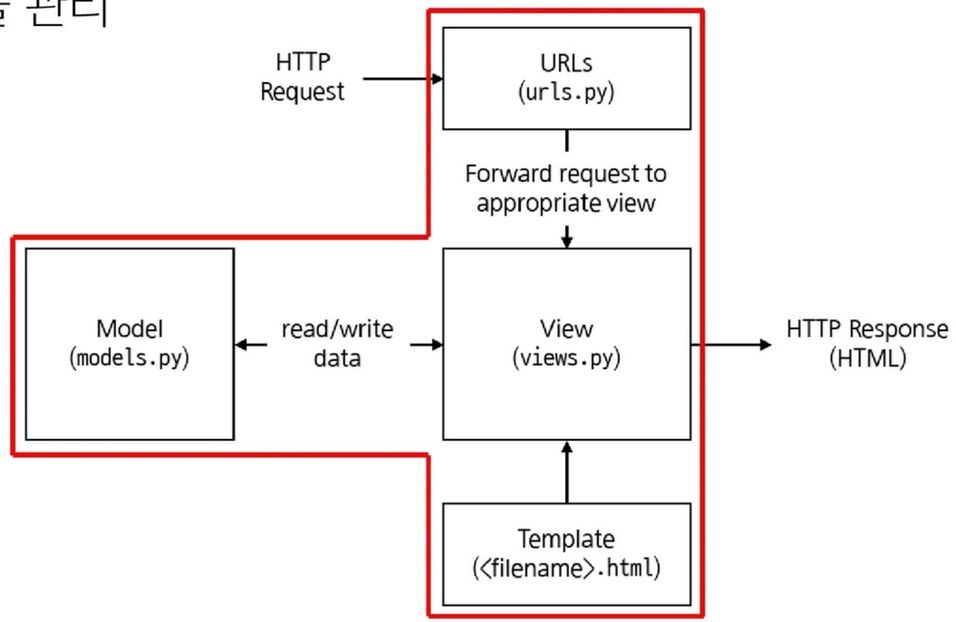
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# MTV 디자인 패턴 정리

# MTV 디자인 패턴 정리 (1/3)

- Model
  - 데이터와 관련된 로직을 관리
  - 응용프로그램의 데이터 구조를 정의하고 데이터베이스의 기록을 관리
- Template
  - 레이아웃과 화면을 처리
  - 화면상의 사용자 인터페이스 구조와 레이아웃을 정의

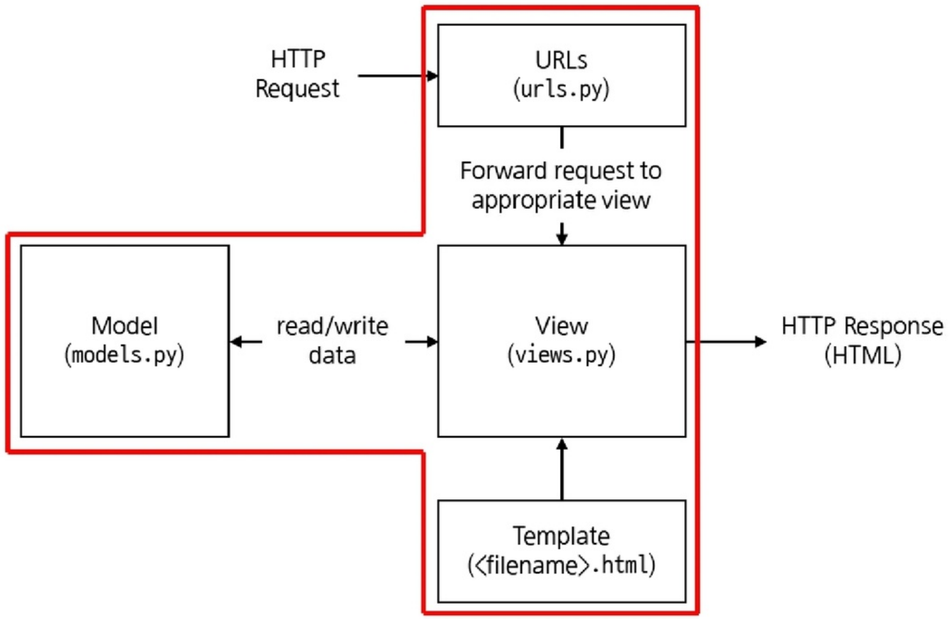



<그림30_Django_Intro & Design Pattern_MTV 패턴>

# MTV 디자인 패턴 정리 (2/3)

- View
  - Model & Template과 관련한 로직을 처리해서 응답을 반환
  - 클라이언트의 요청에 대해 처리를 분기하는 역할

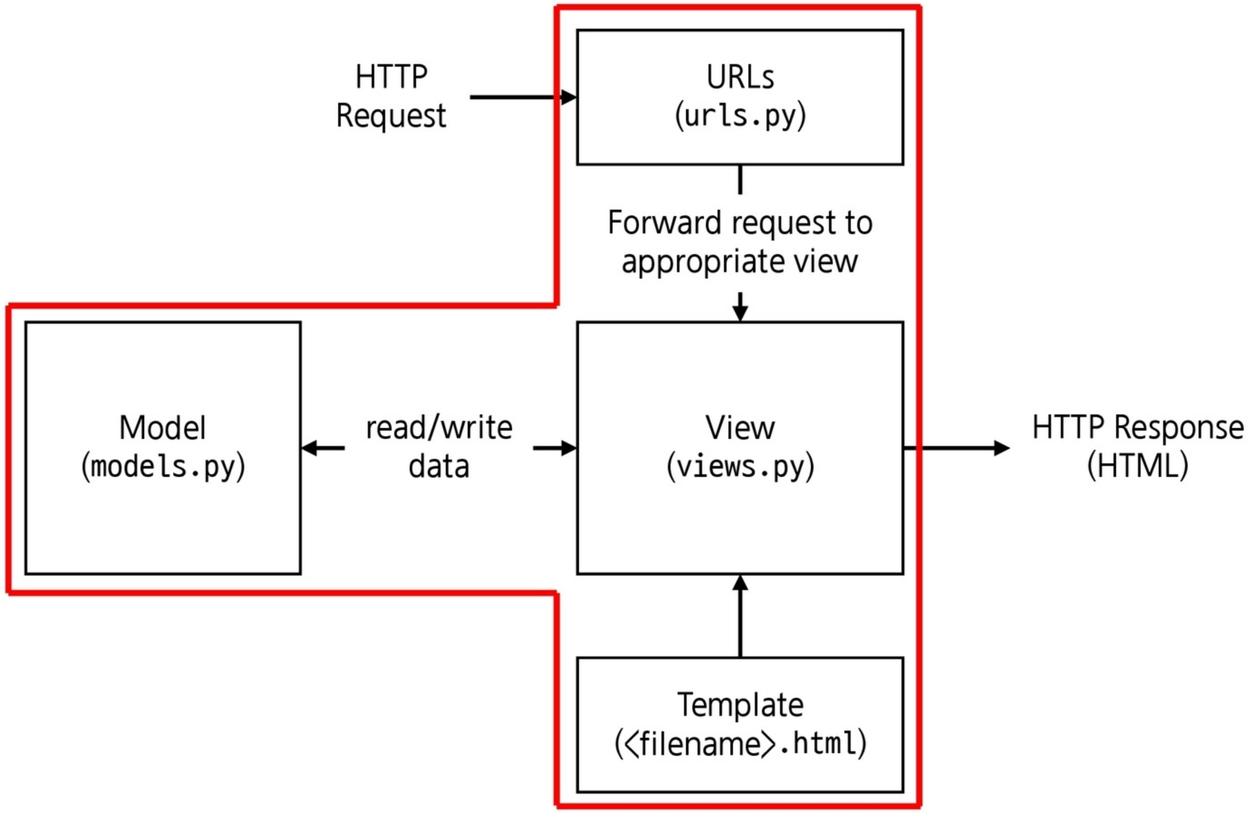
- View 예시
  - 데이터가 필요하다면 Model에 접근해서 데이터를 가져오고,
  - 가져온 데이터를 Template으로 보내 화면을 구성하고,
  - 구성된 화면을 응답으로 만들어 클라이언트에게 반환




<그림30_Django_Intro & Design Pattern_MTV 패턴>

# MTV 디자인 패턴 정리 (3/3)

※ Django가 담당하는 부분




<그림30_Django_Intro & Design Pattern_MTV 패턴>

# 왜 Django는 MTV라고 부를까?

## 장고는 MVC패턴으로 보여집니다, 하지만 컨트롤러를 “뷰”라고 부르고, 뷰를 “템플릿”이라고 합니다. 어떻게 일반적으로 사용하는 이름을 사용하지 않았나요? ¶

음, 기준이 되는 이름은 논쟁의 여지가 있습니다.

저희가 MVC를 해석하기로는, “뷰”는 사용자에게 보여지는 데이터들을 설명합니다. 데이터가 “어떻게” 데이터가 “보이는”지가 아니라 “어떤” 데이터를 보여주는지로 볼 수 있습니다. 뷰는 “당신이 어떤 데이터를 보는지” 를 설명하지 “당신이 어떻게 보는지”를 설명하지 않습니다. 이것은 사소한 차이입니다.

따라서, 우리의 경우 “뷰”는 특정 URL에 대한 파이썬 콜백 함수입니다. 왜냐하면, 콜백함수는 어떤 데이터를 보여주는지 설명하기 때문이죠.

또한, 내용과 표시를 구분하는 것이 현명합니다. 여기서 템플릿이 등장합니다. 장고에서는 “뷰”는 어떤 데이터를 보여줄지를 구현합니다. 그러나 뷰는 일반적으로 템플릿에 어떻게 데이터가 보여질 지를 위임합니다.

그렇다면 “컨트롤러”는 어디에 맞을까요? 장고의 경우, 아마 프레임워크 자체 일것입니다. 프레임워크가 장고 URL설정에 따라 요청을 적절한 뷰에게 전달합니다.

만약 두문자어를 원한다면, 장고를 “MTV” 프레임워크라고 할수 있습니다. “모델”, “템플릿” 그리고 “뷰”로 말이죠. 이렇게 나누는것이 더 적절합니다.

결국 효율적으로 작업을 끝낼 수 있습니다. 그리고 어떻게 이름이 붙여지는지 상관없이 장고는 가장 논리적은 방법으로 일을 처리합니다.

<그림31_Django_Intro & Design Pattern_MTV vs MVC>

https://docs.djangoproject.com/ko/5.2/faq/general/#faq-mtv

# Trailing Comma

## Trailing Comma 정의

- “후행 쉼표”
- 리스트, 딕셔너리, 튜플 등의 자료구조에서 마지막 요소 뒤에 쉼표를 추가하는 것
- 문법적으로 아무런 영향을 주지 않음
- 일반적으로 선택사항
  - 단일 요소 튜플을 만들 때는 예외

## Trailing Comma 사용 이유 (1/2)

- 새로운 요소를 추가하거나 순서를 변경할 때 편리
- 값의 목록, 인자, 또는 import 항목들이 시간이 지남에 따라 확장될 것으로 예상되는 경우에 주로 사용
- 여러 줄에 걸쳐 작성된 데이터 구조에서 유용하며, 코드의 가독성과 유지보수성을 향상시키는데 도움

## Trailing Comma 사용 이유 (2/2)

- 일반적인 패턴은
  - 각 값(등)을 별도의 줄에 배치하고,
  - 항상 후행 쉼표를 추가한 뒤,
  - 닫는 괄호 / 대괄호 / 중괄호를 다음 줄에 배치하는 것
- 닫는 구분 기호와 같은 줄에 후행 쉼표를 두는 것은 권장하지 않음

```
# Correct:
FILES = [
    'setup.cfg’,
    'tox.ini’,
    ]
initialize(FILES,
    error=True,
    )
```

```
# Wrong:
FILES = ['setup.cfg', 'tox.ini',]
initialize(FILES, error=True,)
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 프레임워크의 규칙 및 설계 철학

## 지금까지 등장한 Django의 규칙

1. urls.py에서 각 url 문자열 경로는 반드시 '/'로 끝남
2. views.py에서 모든 view 함수는 첫번째 인자로 요청 객체를 받음
   - 매개변수 이름은 반드시 request로 지정하기
3. Django는 특정 경로에 있는 template 파일만 읽어올 수 있음
   - 특정 경로: app 폴더/templates

## 프레임워크의 규칙

- 프레임워크를 사용할 때는 일정한 규칙을 따라야 하며 이는 저마다의 설계 철학이나 목표를 반영하고 있음
  - 일관성 유지, 보안 강화, 유지보수성 향상, 최적화 등과 같은 이유

- 프레임워크는 개발자에게 도움을 주는 도구와 환경을 제공하기 위해 규칙을 정해 놓은 것이며 우리는 이를 잘 활용하여, 특정 기능을 구현하는 방법을 표준화하고 개발 프로세스를 단순화할 수 있도록 해야함

**TIP**

**Django의 설계 철학 문서를 찾아보세요!**

- https://docs.djangoproject.com/ko/5.2/misc/design-philosophies/

# 실습



## Django 프로젝트

- 3219. requirements 활용하기
- 3220. 현재 위치에 프로젝트 생성하기
- 3221. 한국어 프로젝트 생성하기
- 3019. 프로젝트 생성

## Django Design Pattern

- 2641. 장고 프로젝트 시작 예제 1
- 2642. 장고 프로젝트 시작 예제 2
- 3020. 메인페이지 생성
- 3021. 추가 페이지 생성

# 확인 문제



## Q 각 문제에 올바른 답안을 생각해봅시다

**1** 클라이언트와 서버 구조에서, 사용자의 요청을 받아 처리하는 역할을 하는 것은?

a) 클라이언트
b) 서버
c) 프레임워크
d) 데이터베이스

**2** 웹 프레임워크의 주요 목적은?

a) 웹 서버를 구축한다
b) 웹 개발에 필요한 코드를 줄이고 구조화를 돕는다
c) 운영 체제를 만든다
d) 하드웨어를 제어한다

**3** Django가 포함하는 구성요소 중 하나가 아닌 것은?

a) Models
b) Templates
c) Views
d) Compilers

**4** 가상 환경을 사용하는 이유는?

a) 컴퓨터의 메모리를 늘리기 위해
b) 특정 프로젝트마다 독립적인 파이썬 패키지를 관리하기 위해
c) 웹 브라우저 성능을 높이기 위해
d) 인터넷 속도를 빠르게 하기 위해

Test

# 확인 문제



## Q 각 문제에 올바른 답안을 생각해봅시다

**5** 다음 중 Django 프로젝트 생성 명령어로 올바른 것은?

a) `django create project`
b) `python startproject djangoapp`
c) `django-admin startproject myproject`
d) `pip install project`

**6** 소프트웨어 설계 시 반복적으로 나타나는 문제를 해결하기 위한 일반적인 방법을 무엇이라 하는가?

a) 알고리즘
b) 변수
c) 디자인 패턴
d) 프레임워크

**7** Django에서 'App'의 역할로 적절한 것은?

a) 전체 서버를 운영한다
b) 웹사이트 전체 구조를 담당한다
c) 하나의 기능 또는 모듈 단위를 구성한다
d) Python 인터프리터를 실행한다

**8** venv 모듈을 이용해 가상환경을 생성하는 명령어는?

a) `python install venv`
b) `python manage.py runserver`
c) `python -m venv myenv`
d) `pip install venv create`

# 확인 문제



## Q 각 문제에 올바른 답안을 생각해봅시다

**9** 다음 중 Django의 주요 특징으로 올바르지 않은 것은?

a) MTV 디자인 패턴을 기반으로 한다

b) 보안 기능이 내장되어 있다

c) 고정된 HTML 템플릿만 사용 가능하다

d) ORM(Object Relational Mapping)을 제공한다

**10** Django에서 `render()` 함수의 주된 목적은 무엇인가?

a) URL을 등록한다

b) 정적 파일을 저장한다

c) 데이터를 템플릿에 전달하여 응답을 반환한다

d) 데이터베이스를 초기화한다

**11** 새로 만든 Django 앱을 프로젝트에 등록하기 위해 설정해야 하는 파일은?

a) `urls.py`

b) `settings.py`

c) `views.py`

d) `models.py`

**12** MTV 패턴에서 사용자에게 보여지는 화면을 구성하는 요소는 무엇인가?

a) Model

b) Template

c) View

d) URLconf

# 확인 문제



## 정답 및 해설

1. b) 서버 2. b) 웹 개발에 필요한 코드를 줄이고 구조화를 돕는다 3. d) Compilers
4. b) 특정 프로젝트마다 독립적인 파이썬 패키지를 관리하기 위해 5. c) django-admin startproject myproject
6. c) 디자인 패턴 7. c) 하나의 기능 또는 모듈 단위를 구성한다 8. c) python -m venv myenv

1. 서버는 클라이언트로부터 요청을 받고 처리한 결과를 클라이언트로 전달하는 역할을 합니다.
2. 웹 프레임워크는 웹 애플리케이션 개발을 효율적으로 할 수 있도록 미리 정의된 구조와 기능을 제공합니다.
3. Django는 MTV 패턴(MTV: Model, Template, View)을 기반으로 하며, Compiler는 포함되지 않습니다.
4. 가상 환경은 프로젝트별로 필요한 파이썬 패키지를 격리하여 관리할 수 있도록 도와줍니다.
6. Django 프로젝트는 'django-admin startproject 프로젝트이름' 명령어로 생성합니다.
7. 디자인 패턴은 자주 마주치는 설계 문제를 해결하기 위한 표준화된 해결 방법입니다.
8. Django App은 특정 기능을 담당하는 코드 집합으로, 프로젝트 내에서 여러 개의 App을 운영할 수 있습니다.
5. 'python -m venv 환경이름' 명령어로 파이썬 가상환경을 생성할 수 있습니다.

# 확인 문제



## 정답 및 해설

9. c) 고정된 HTML 템플릿만 사용 가능하다 10. c) 데이터를 템플릿에 전달하여 응답을 반환한다
11. b) `settings.py` 12. b) Template

9. Django는 다양한 템플릿 엔진을 사용할 수 있으며, HTML 템플릿에 고정되지 않습니다.
10. `render()` 함수는 요청, 템플릿 이름, `context` 데이터를 받아 템플릿을 렌더링한 HTTP 응답을 반환합니다.
11. `settings.py`의 `INSTALLED_APPS` 항목에 앱 이름을 추가해야 Django가 해당 앱을 인식하고 사용합니다.
12. Template은 사용자에게 보여질 웹페이지 화면을 정의하며 HTML로 작성됩니다.

# 활동 정리



## 핵심 키워드

| 개념 | 설명 | 예시 |
|---|---|---|
| 클라이언트 (Client) | 서비스를 요청하는 주체 | 웹 브라우저, 모바일 앱 |
| 서버 (Server) | 클라이언트의 요청에 응답하는 주체 | 웹 서버, 데이터베이스 서버 |
| 가상 환경 | 하나의 컴퓨터 안에서 또 다른 독립된 파이썬 환경 | |
| 웹 프레임워크 | 웹 애플리케이션을 빠르게 개발할 수 있도록 도와주는 도구 | Django, Spring, Express |
| Django | Python 기반의 대표적인 웹 프레임워크 | |
| 디자인 패턴 (Design Pattern) | 공통적인 문제를 해결하는 데 쓰이는 형식화 된 관행 | MVC, MTV |

<표2_Django_Intro & Design Pattern_핵심 키워드>

# 활동 정리



## 오늘 공부한 내용 요약 및 정리

- Web Application (Web Service)
  - 인터넷을 통해 사용자에게 제공되는 소프트웨어 프로그램
  - 다양한 디바이스를 통해 접근하고 사용할 수 있음

- 클라이언트 - 서버 구조
  - 인터넷에 연결된 서로 다른 두 컴퓨터가 데이터를 주고받는 웹의 동작 방식 중 하나
    - 클라이언트(Client) - 서비스를 요청하는 주체
    - 서버(Server) - 클라이언트의 요청에 응답하는 주체
  - 클라이언트가 인터넷에 연결된 서버에게 요청을 보내면, 서버는 요청에 알맞은 데이터 또는 파일을 응답하는 형태

# 활동 정리



## 오늘 공부한 내용 요약 및 정리

- Frontend와 Backend
  - Frontend: 사용자 인터페이스를 구성하고, 사용자가 애플리케이션과 상호작용할 수 있도록 함
    - HTML, CSS, JavaScript, 프론트엔드 프레임워크 등
  - Backend: 서버 측에서 동작하며, 클라이언트의 요청에 대한 처리와 데이터베이스 상호작용을 담당
    - 서버 언어, 백엔드 프레임워크, 데이터베이스, API 등

- 웹 프레임워크
  - 웹 애플리케이션을 빠르게 개발할 수 있도록 도와주는 도구
  - 개발에 필요한 기본 구조, 규칙, 라이브러리 등을 제공
  - 대표적인 웹 프레임워크로 Django가 있음

# 활동 정리



## 오늘 공부한 내용 요약 및 정리

- 가상 환경
  - 하나의 컴퓨터 안에서 또 다른 독립된 파이썬 환경
  - 프로젝트별로 라이브러리를 독립적으로 관리하게 해주어 의존성 충돌을 방지
  - 팀원들 끼리 같은 버전의 라이브러리를 사용할 수 있도록 하여 에러 가능성을 줄여줌

- 디자인 패턴
  - 소프트웨어 설계에서 발생하는 문제를 해결하기 위한 일반적인 해결책이자
  - 공통적인 문제를 해결하는 데 쓰이는 형식화 된 관행
  - 대표적으로 MVC 디자인 패턴이 있음
    - 데이터 - 사용자 인터페이스 - 비즈니스 로직을 분리
  - MTV 디자인 패턴은 기존 MVC에서 Django의 구조에 맞춰 명칭을 다르게 정의한 것

# 활동 정리



## 오늘 공부한 내용 요약 및 정리

- Django project & Django application
  - 프로젝트(project): 전체 웹 애플리케이션의 설정 등을 관리
  - 애플리케이션(application): 독립적으로 작동하는 기능 단위 모듈
  - 프로젝트는 애플리케이션의 집합

- Django에서의 요청과 응답
  1. 브라우저가 URL로 요청
  2. 요청 주소와 일치하는 view 함수 호출
  3. view 함수가 응답 객체 생성 및 반환
  4. 브라우저는 view 함수가 반환한 응답 객체를 해석해 페이지를 렌더링

# 활동 정리



## 서버가 있다면 웹 페이지를 브라우저에 보내줄 수 있습니다.

- 웹 프레임워크인 Django를 활용하여 서버를 만들 수 있습니다.
- 웹 브라우저로 Django 서버에 요청을 하면, 서버가 HTML을 응답하게 할 수 있습니다.

1. 웹 브라우저는 클라이언트로서 인터넷 상의 컴퓨터에 요청을 보냅니다.
2. 이 요청을 받은 서버는 요청에 따라 알맞은 응답을 보냅니다.
3. 이러한 형태로 제공되는 서비스를 웹 애플리케이션이라고 하며, 이를 빠르게 개발할 수 있도록 도와주는 도구가 웹 프레임워크 입니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

# 감사합니다